ECOLE POLYTECHNIQUE DE BRUXELLES

Rapport du projet de TELECOM

ELEC-H304 - Physique des télécommunications

Développement d'un algorithme de Ray-Tracing

Auteurs :

Erik Baecke

Daniel Usovitch

Titulaire du cours :

Philippe De Doncker

Assistant du cours :

Quentin Gontier

Année académique 2022-2023

Table des matières

1 Hypothèses **2**

2 Validation de l'exercice 1 du TP 8 **3**

3 Résultats du calcul **9**

4 Fonctionnalités du code et optimisations **14**

5 Utilisation du GUI Matlab **17**

A Annexe : Code **19**
A.1 Code principal 19
A.2 Fonctions 23
A.3 Code du GUI Matlab 32

1 Hypothèses

Puisqu'il est impossible de décrire parfaitement la réalité dans un code informatique, il est impératif de définir un modèle de la situation. Ce modèle est basé sur les hypothèses qui sont détaillées dans cette section.

1. Lorsqu'une onde arrive à un mur, 2 situations sont possibles : soit elle est réfléchie, soit elle est transmise. Ici, la diffraction est négligée.

2. Dans le cas d'une réflexion sur un mur, l'onde repart avec le même angle par rapport à la normale du mur. L'angle incident et réfléchi sont donc identiques. De plus, l'intensité du champ électrique porté par l'onde est réduite d'un facteur Γ_m

3. Dans le cas d'une transmission, l'onde traverse le mur avec un angle différent par rapport à l'angle incident, décrit par l'équation de Snell-Descartes : L'onde ressort du mur avec le même angle qu'à l'entrée. Comme pour la réflexion, l'intensité du champ électrique porté par l'onde est réduite d'un facteur T_m

4. Les coefficients de réflexion et de transmission prennent en compte les réflexions et transmissions successives à l'intérieur du mur.

5. L'épaisseur des murs n'est prise en compte que pour le calcul des coefficients, c'est-à-dire que dans le tracé des rayons, l'onde traverse le mur sans changement de trajectoire du à la loi de Snell-Descartes aux deux surfaces du mur. Si c'était le cas, le rayon aurait la même inclinaison avant et après le mur mais serait discontinu au niveau du mur.

6. Tout se fait bidimensionnellement, les émetteurs et récepteurs se trouvent à la même hauteur et les rayons ne se réfléchissent ni sur le sol, ni sur le plafond.

7. Puisqu'il est impossible de calculer la puissance en chaque point (il y en a une infinité), la formule de la puissance moyenne sera utilisée au lieu de la puissance locale. Cela permet également de ne pas avoir de répartition de puissance variant rapidement dans l'espace dû aux interférences constructives et destructives.

8. On prend en compte une zone d'exclusion de rayon de 50 cm autour des sources émettrices.

9. Puisque les gains de TX1 et TX2 ne sont pas spécifiés, le gain est le gain maximal. Puisque ce sont des émetteurs $\lambda/2$ ce dernier vaut 2,14 dBi soit environ 1,64.

2 Validation de l'exercice 1 du TP 8

L'objectif de cet exercice est de développer un algorithme de Ray-Tracing permettant de modéliser la propagation des ondes électromagnétiques, de calculer la puissance reçue dans une configuration spécifique et de comparer les résultats obtenus avec des données théoriques,tout en utilisant le calcul vectoriel de la méthode des images.

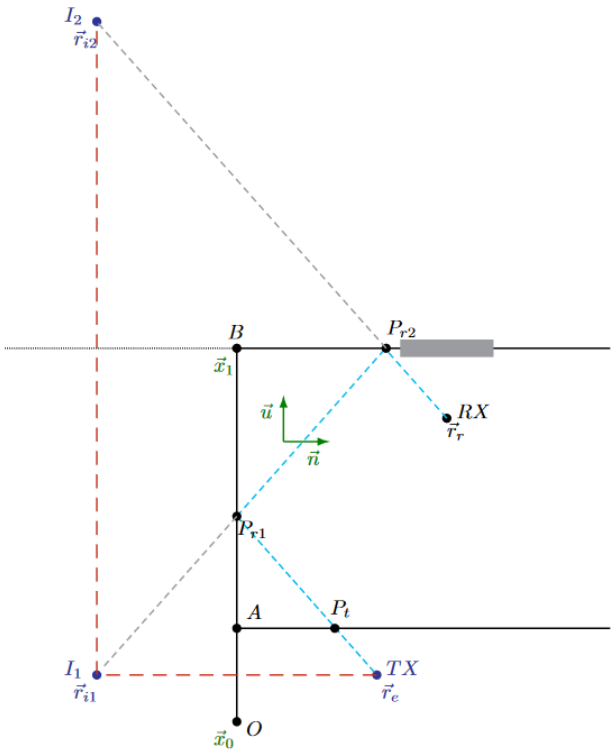
Dans le cadre de cet exercice, la configuration étudiée est celle d'un garage présentant deux murs horizontaux et un mur vertical. Afin de générer des rayons dans l'espace, il est nécessaire de déterminer les positions des images associées à chaque mur, permettant ainsi de simuler les réflexions sur l'ensemble des surfaces de la pièce et de calculer les coefficients de réflexion et de transmission aux interfaces entre les différents matériaux présents dans la configuration. A partir de ces résultats, il est alors possible de calculer les champs électriques à l'aide de la loi de Friss et d'évaluer ainsi les puissances reçues en tout point à l'intérieur du garage. Afin de réaliser cela, nous procéderons à des calculs uniquement pour un rayon comprenant deux réflexions, puis nous comparerons ces résultats avec ceux obtenus lors de la simulation.

Tout d'abord, commençons par la réflexion au point P_{r1}. Avant tout, il est nécessaire de déterminer l'image I_1 de l'émetteur.

$$\vec{r_1} = \vec{r_e} - 2\langle \vec{r_e}, \vec{n} \rangle \vec{n} = (-32, 10) \tag{1}$$

Ensuite, on détermine l'image I_2 de I_1. En procédant ainsi de la même manière, on obtient :

$$\vec{r_2} = (-32, 150) \tag{2}$$



Désormais, nous sommes en mesure de calculer les vecteurs $\vec{d_1}$ et $\vec{d_2}$.

$$\vec{d_1} = \vec{r}x - \vec{r_2} = (79, -85) \tag{3}$$

Nous constatons que le vecteur $\vec{d_1}$ intercepte le mur vertical. Par conséquent, en utilisant le produit scalaire, nous pouvons déterminer $\cos\theta_\mathrm{i}$ en utilisant la normale du mur.

$$\cos\theta_\mathrm{i} = |\langle \frac{\vec{d_1}}{\|\vec{d_1}\|}, \vec{u} \rangle| = 0.732 \tag{4}$$

$$\sin\theta_\mathrm{i} = \sqrt{1 - \cos\theta_\mathrm{i}^2} = 0.681 \tag{5}$$

En appliquant la loi de Snell, nous pouvons déterminer $\sin\theta_\mathrm{t}$ et $\cos\theta_\mathrm{t}$.

$$\sin\theta_\mathrm{t} = \sqrt{\frac{1}{\epsilon_\mathrm{r}}} \sin\theta_\mathrm{i} = 0.31 \tag{6}$$

$$\cos\theta_\mathrm{t} = \sqrt{1 - \sin\theta_\mathrm{t}^2} = 0.95 \tag{7}$$

$$s = 0.155m \tag{8}$$

$$\Gamma_\perp(\theta_i) = \frac{Z_m \cos\theta_i - Z_0 \cos\theta_t}{Z_m \cos\theta_i + Z_0 \cos\theta_t}$$

$$\Gamma_\perp(\theta_\mathrm{i}) = -0.4803 + 0.0149i \tag{9}$$

$$\Gamma_m(\theta_i) = \Gamma_\perp(\theta_i) - \left(1 - \Gamma_\perp^2(\theta_i)\right) \frac{\Gamma_\perp(\theta_i)\, e^{-2\gamma_m s}\, e^{j\beta 2s \sin\theta_t \sin\theta_i}}{1 - \Gamma_\perp^2(\theta_i)\, e^{-2\gamma_m s}\, e^{j\beta 2s \sin\theta_t \sin\theta_i}} =$$

$$\Gamma_\mathrm{m}(\theta_\mathrm{i}) = -0.418 + 0.246i \tag{10}$$

Il reste maintenant à déterminer la position de $P_{\mathrm{r}2}$ et le coefficient de réflexion en ce point.

$$P_r \equiv \vec{x}_0 + t\,\vec{u}.$$

où

$$t = \frac{d_y\,(r_{ix} - x_0) - d_x\,(r_{iy} - y_0)}{u_x d_y - u_y d_x}$$

$$t = 33.06 \tag{11}$$

$$P_{r2} = (33.058; 80) \tag{12}$$

Déterminons le prochain point de réflexion P_{r1} et le coefficient de réflexion en ce point.

$$\vec{d_2} = \vec{P_{r2}} - \vec{r_1} \tag{13}$$

$$\cos\theta_i = |\langle \frac{\vec{d_2}}{\|\vec{d_2}\|}, \vec{n} \rangle| = 0.680 \tag{14}$$

$$\sin\theta_i = \sqrt{1 - \cos\theta_i{}^2} = 0.7325 \tag{15}$$

$$\sin\theta_t = \sqrt{\frac{1}{\epsilon_r}} \sin\theta_i = 0.3343 \tag{16}$$

$$\cos\theta_t = \sqrt{1 - \sin\theta_t{}^2} = 0.9425 \tag{17}$$

$$s = 0.155m \tag{18}$$

$$P_r \equiv \vec{x}_0 + t\,\vec{u}.$$

où

$$t = \frac{d_y\,(r_{ix} - x_0) - d_x\,(r_{iy} - y_0)}{u_x d_y - u_y d_x}$$

$$t = 44.4304 \tag{19}$$

$$P_{r1} = (0; 44.4304) \tag{20}$$

$$\Gamma_\perp(\theta_i) = \frac{Z_m \cos\theta_i - Z_0 \cos\theta_t}{Z_m \cos\theta_i + Z_0 \cos\theta_t}$$

$$\Gamma_\perp(\theta_i) = -0.5048 + 0.0144i \tag{21}$$

$$\Gamma_m(\theta_i) = \Gamma_\perp(\theta_i) - \left(1 - \Gamma_\perp^2(\theta_i)\right) \frac{\Gamma_\perp(\theta_i)\, e^{-2\gamma_m s}\, e^{j\beta 2s \sin\theta_t \sin\theta_i}}{1 - \Gamma_\perp^2(\theta_i)\, e^{-2\gamma_m s}\, e^{j\beta 2s \sin\theta_t \sin\theta_i}} =$$

$$\Gamma_m(\theta_i) = -0.4713 + 0.2520i \tag{22}$$

Calculons le coefficient de transmission T_m.

$$T_m(\theta_i) = \frac{\left(1 - \Gamma_{\perp}^2(\theta_i)\right)\, e^{-\gamma_m s}}{1 - \Gamma_{\perp}^2(\theta_i)\, e^{-2\gamma_m s}\, e^{j\beta 2s \sin\theta_t \sin\theta_i}}$$

$$T_{\mathrm{m}}(\theta_{\mathrm{i}}) = 0.6297 + 0.0900i \tag{23}$$

Il nous reste à calculer le champ électrique et la puissance en watts ainsi qu'en dBm. En utilisant la loi de Friss :

$$\underline{E_n} = \Gamma_1\Gamma_2\Gamma_3... \; T_1T_2T_3... \; D_1D_2D_3... \; \sqrt{60\; G_{TX}(\theta_{TXn}\phi_{TXn})\; P_{TX}}\; \frac{e^{-j\beta d_n}}{d_n};$$

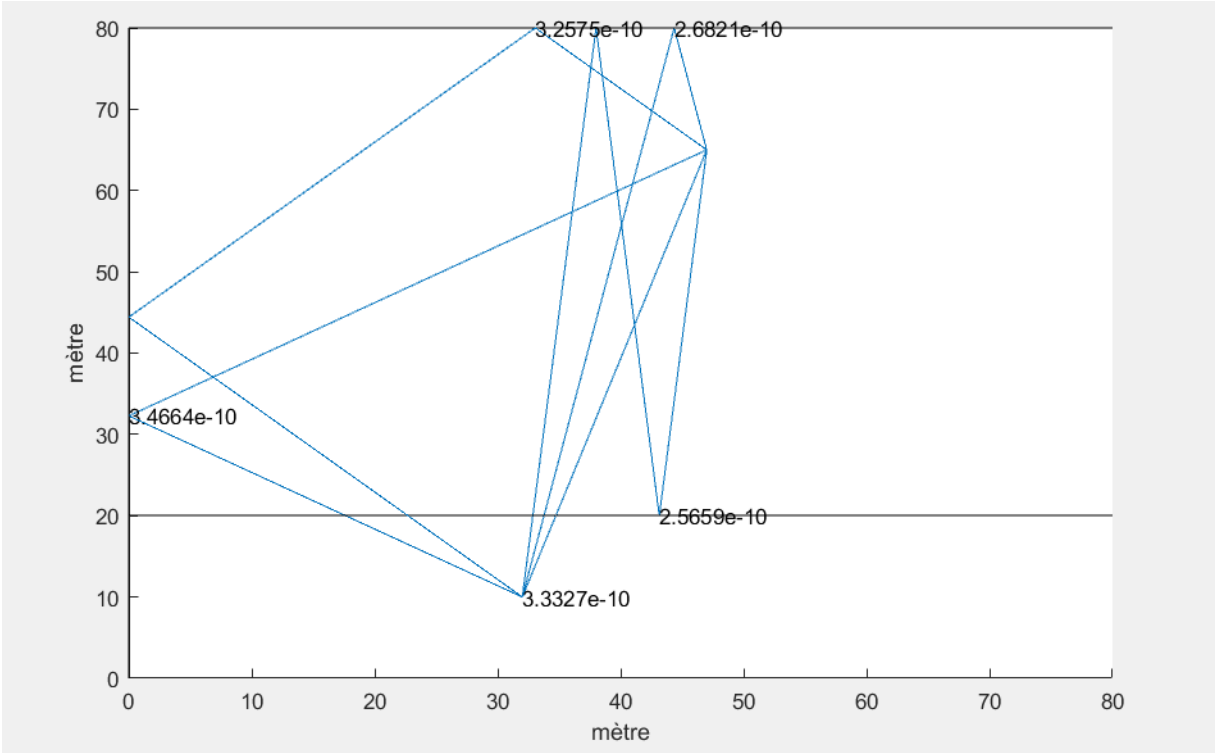
En vertu de l'hypothèse 1, nous négligeons les coefficients de diffraction. Ainsi, nous obtenons le champ électrique.

$$E = 0,000446 - 2,11318753770329e - 05i \tag{24}$$

Enfin, en prenant compte de la puissance d'onde directe, une réflexion ainsi que deux réflexions, la puissance reçue par le récepteur RX est connue.

$$Prx = 3,2565.10^{-10}\text{Watt} = -64,9\text{dBm} \tag{25}$$

Maintenant, il est nécessaire de vérifier, à l'aide de la simulation, si les données générées sont identiques à celles des résultats obtenus.



Type d'onde	Puissances théorique (Watt)	Puissance simulation (Watt)	dBm
onde directe	$3{,}33.10^{-10}$	$3{,}3327.10^{-10}$	-64,77dBm
Une réflexion	$2{,}69.10^{-10}$	$2{,}682.10^{-10}$	-65.72 dBm
Deux réflexions	$3{,}2565.10^{-10}$	$3{,}257.10^{-10}$	-64,9 dBm
Deux réflexions	$2{,}57.10^{-10}$	$2{,}5659.10^{-10}$	-65,91 dBm

La conclusion est que le garage s'ouvrira en prenant uniquement en compte l'onde directe, car la puissance totale dépasse le seuil de sensibilité. Cependant, la prise en compte d'autres ondes entraîne des interférences destructives qui diminuent la puissance totale au niveau du récepteur.

Le graphiques résultant de l'algorithme ci-dessus confirme que les puissances en watt sont conformes à celles qui ont été calculées précédemment. Ainsi, l'algorithme permet d'obtenir des résultats satisfaisants sans avoir d'erreur de calcul. À cet égard, voici deux résultats détaillant la répartition de la puissance en dBm dans le garage :

— La répartition de la puissance en dBm dans tous les points de l'espace à l'intérieur du garage :

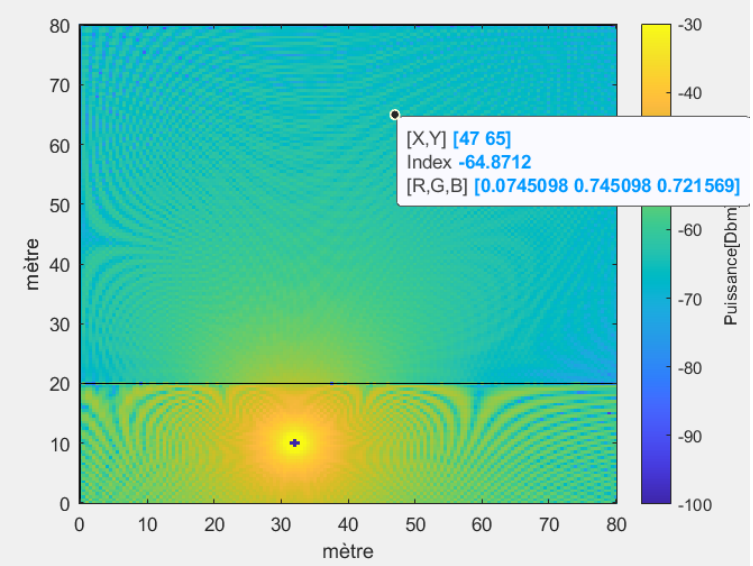


FIGURE 1 – Puissance reçue en tout point de l'espace en dBm avec un carré de 0.5x0.5

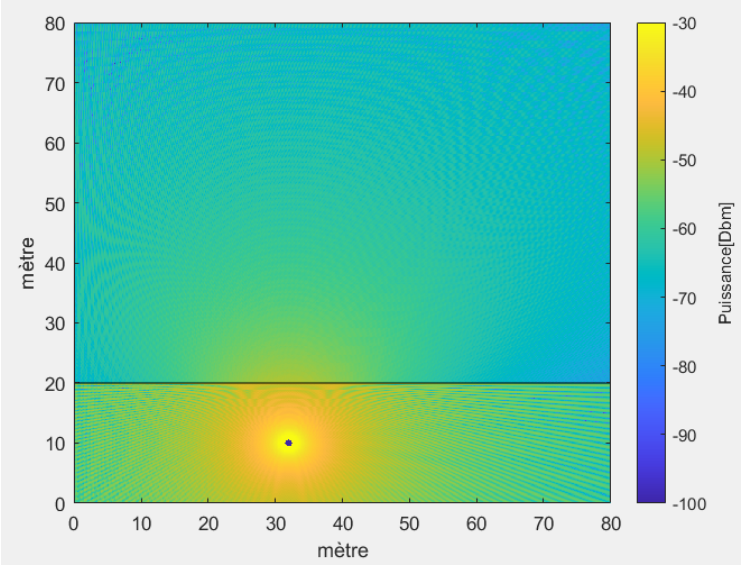


FIGURE 2 – Puissance reçue en tout point de l'espace en dBm avec un carré de 0.1x0.1

En conclusion, la puissance en dBm de la simulation reçue par le récepteur RX est similaire à celle prédite théoriquement. Par conséquent, l'algorithme de ray-tracing fonctionne de manière optimale et génère des données cohérentes qui correspondent parfaitement aux résultats théoriques obtenus. Désormais, l'algorithme est opérationnel et prêt à être appliqué dans une configuration plus complexe, similaire à celle de l'usine Meet-A du projet.

3 Résultats du calcul

En adaptant l'algorithme de l'exercice 1 du TP8, la configuration a été modifiée pour inclure plus de murs et les propriétés des matériaux ont été modifiées. Les murs en briques ont une épaisseur de 30 cm, les murs blancs sont des cloisons d'une épaisseur de 10 cm, et les murs en béton ont une épaisseur de 50 cm. Les propriétés des matériaux, telles que les conductivités thermiques et les permittivités relatives, ont été spécifiées pour chaque type de mur. Ces modifications permettent une modélisation plus précise de la propagation des ondes électromagnétiques à travers les différents matériaux présents dans l'usine de Meet-A. Voici un plan illustratif de l'usine :

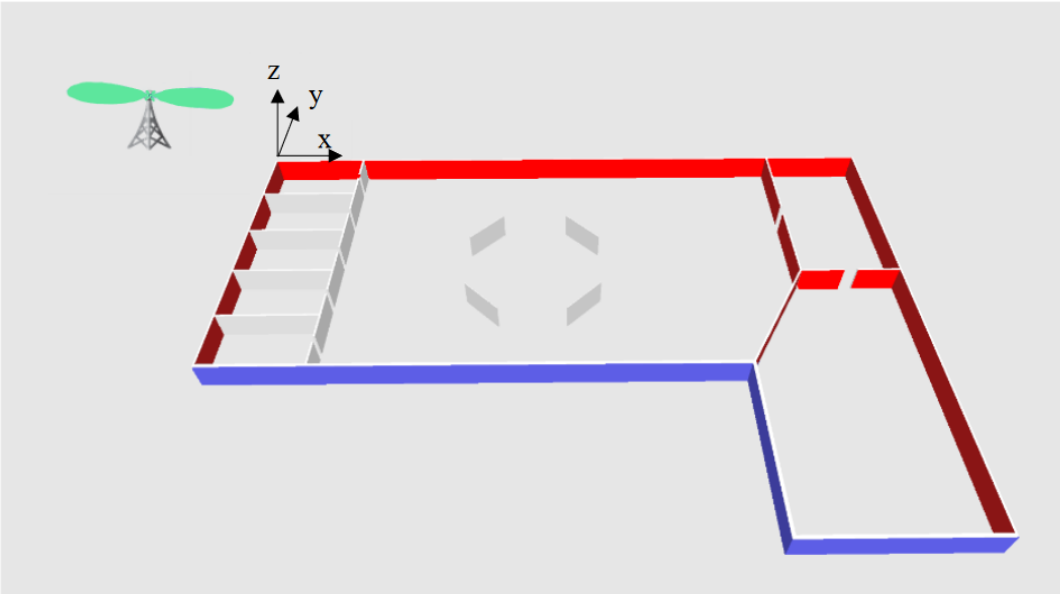


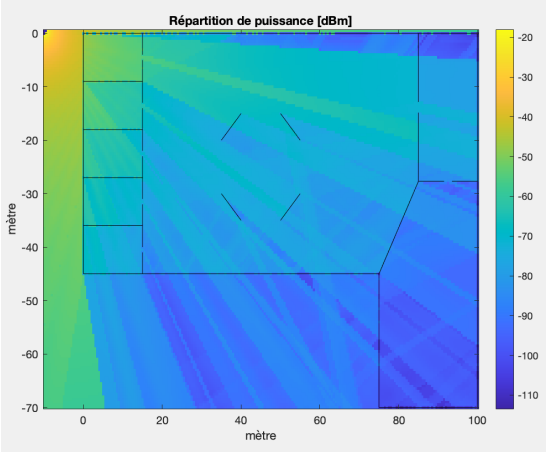
FIGURE 3 – Plan de l'usine Meet-A

Les murs rouges sont des murs en briques d'une épaisseur de 30 cm, les murs blancs sont des cloisons d'une épaisseur de 10 cm, et les murs bleus sont des murs en béton d'une épaisseur de 50 cm.

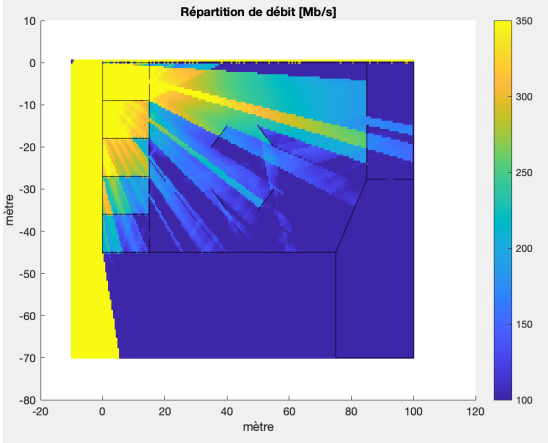
Ensuite, pour obtenir une bonne répartition de puissance et de débit internet dans toute l'usine. Cela est d'abord fait avec une antenne émettrice 5G à 26GHZ avec une puissance d'émission de 35dBm TX2 puis pour TX3 dont le gain varie en fonction de la direction des rayons émis selon la formule suivante :

$$G(\theta)[dB] = G_{\max}[dB] - 12(\frac{\phi - \delta}{\phi_{3\text{dB}}})^2 \tag{26}$$

Tout d'abord, il est crucial de procéder à un test avec l'émetteur TX2 dont le gain de directivité est de 1.64, afin d'obtenir les niveaux de puissance et de débit dans l'usine. Voici les résultats obtenus lors de ce test :

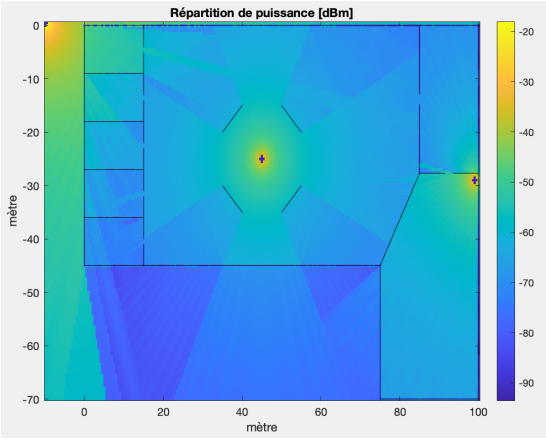


(a) Répartition de puissance en dBm pour un émetteur de type TX2

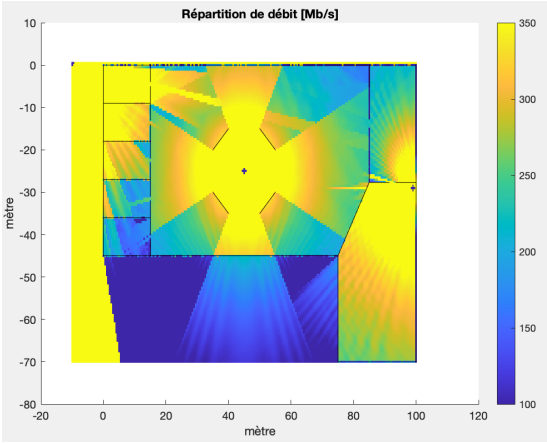


(b) Répartition de débit internet en Mb/s pour un émetteur de type TX2

Les graphiques ci-dessus montrent bien que la puissance et le débit du réseau ne sont pas bien répartis. Par conséquent, il est obligatoire de placer des émetteurs de type TX1 supplémentaires à l'intérieur de l'usine afin d'assurer la couverture optimale de réseau au sein du bâtiment. Pour ce faire, il faut placer ces émetteurs dans des zones en faible d'intensité. Ainsi, cela donne les résultats suivants :

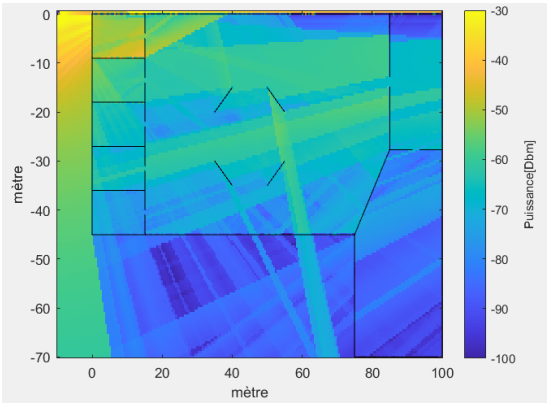


(a) Répartition de puissance en dBm pour un émetteur de type TX2

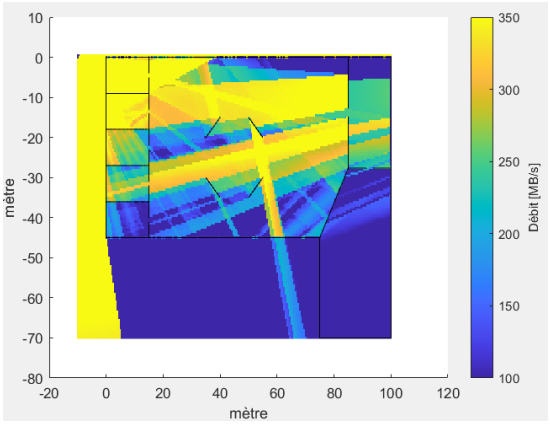


(b) Répartition de débit internet en Mb/s pour un émetteur de type TX2

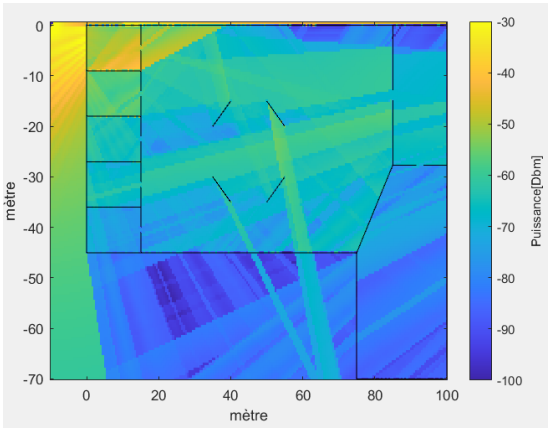
Pour optimiser la répartition de la puissance pour un émetteur de type TX3 cette fois-ci, il est important de varier l'angle δ en vue d'assurer une couverture adéquate sur l'ensemble de l'espace de la pièce. Après plusieurs tests, il a été constaté que l'angle adéquat est de 0.78 radians. Voici les différents tests effectués pour différentes valeurs de δ pour un émetteur de type TX3 :



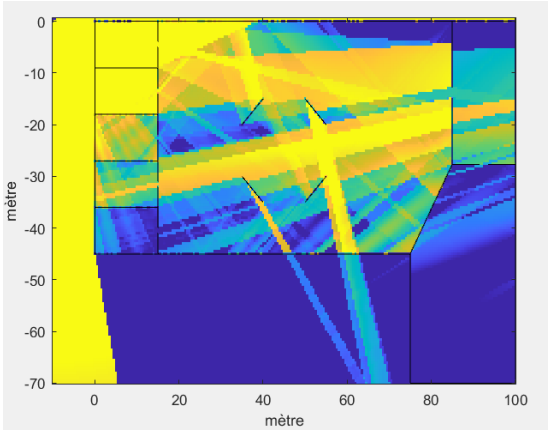
(a) Répartition de puissance en dBm avec $\delta = 0.1$ rad pour un émetteur de type TX3



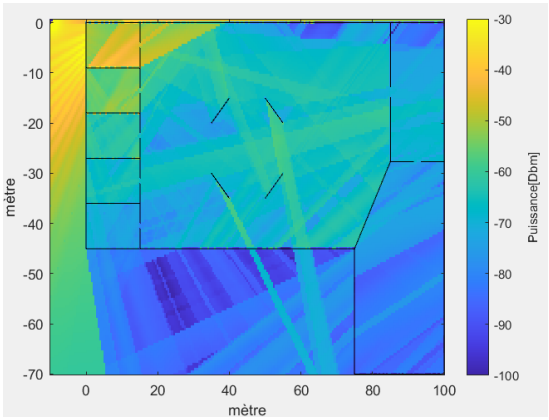
(b) Répartition de débit internet en Mb/s avec $\delta = 0.1$ rad pour un émetteur de type TX3



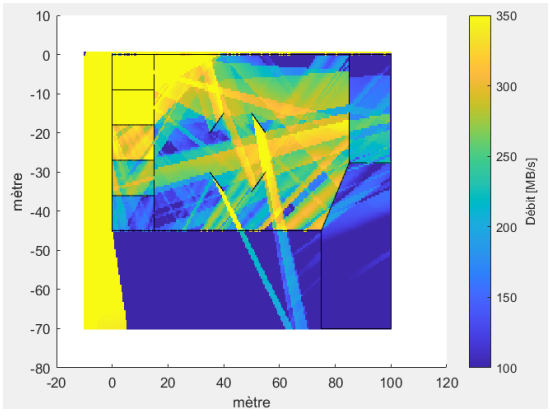
(a) Répartition de puissance en dBm avec $\delta = 0.3$ rad pour un émetteur de type TX3



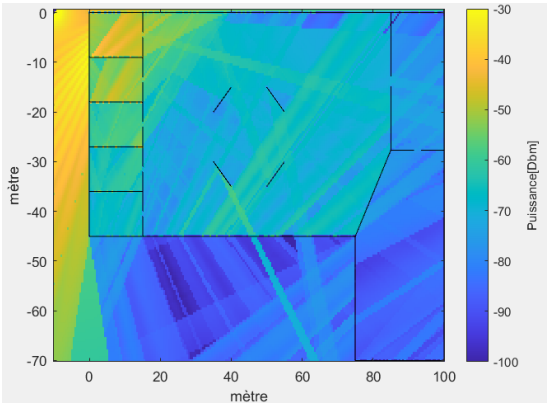
(b) Répartition de débit internet en Mb/s avec $\delta = 0.3$ rad pour un émetteur de type TX3



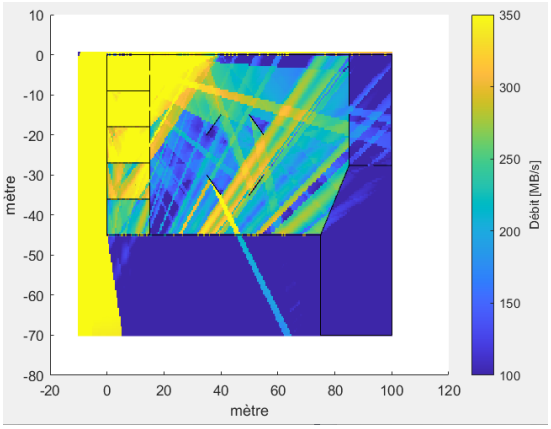
(a) Répartition de puissance en dBm avec $\delta = 0.5$ rad pour un émetteur de type TX3



(b) Répartition de débit internet en Mb/s avec $\delta = 0.5$ rad pour un émetteur de type TX3

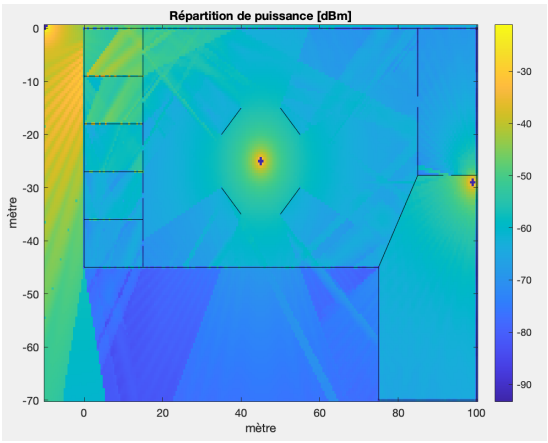


(a) Répartition de puissance en dBm avec $\delta = 0.78$ rad pour un émetteur de type TX3

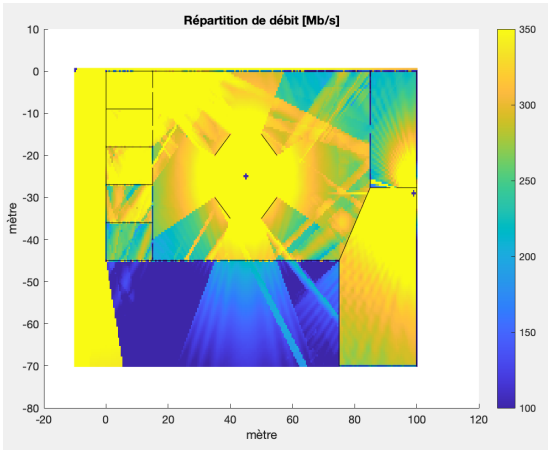


(b) Répartition de débit internet en Mb/s avec $\delta = 0.78$ rad pour un émetteur de type TX3

Pour un delta de 0.78 radians, il est observé que la puissance et le débit sont bien répartis à gauche et moyennement répartis au centre de l'usine. Cependant, la partie droite présente une faible intensité de puissance et donc un faible débit du réseau. Il est donc prévu de placer un émetteur de type TX1 dans cette partie de l'usine ainsi qu'au centre. Il a été suggéré de mettre un gain directivité de 1 pour le l'émetteur TX1. Voici, les résultats obtenus :



(a) Répartition de puissance en dBm avec deux émetteurs TX1



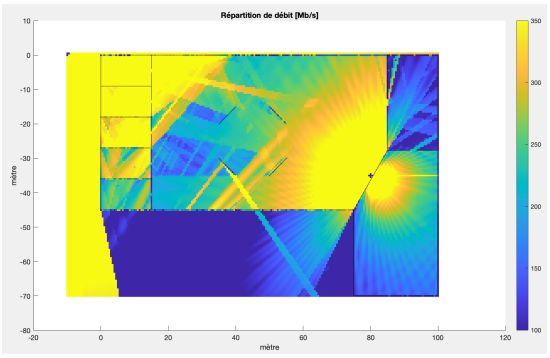
(b) Répartition de débit internet en Mb/s avec deux émetteurs TX1

Via les deux graphiques ci-dessus, il est clair qu'il y a une bonne répartition de la puissance et du débit Internet dans toute l'usine, offrant ainsi une connexion stable et performante aux robots. Les résultats dépassent même les 100 Mb/s, ce qui démontre l'importance de placer les deux émetteurs TX1. Par ailleurs, l'émetteur TX3 offre un meilleur résultat en terme de performance de connexion par rapport à l'émetteur TX2 dont il était démontré précédemment.

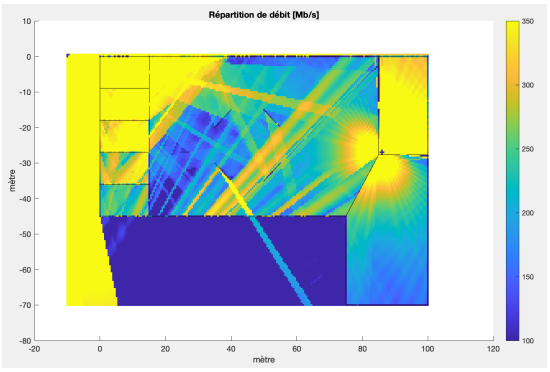
En conclusion, les émetteurs TX1 et TX3 offrent une excellente répartition de débit dans toute l'usine, dépassant les 100 MB/s. Par ailleurs, la puissance reçue au sein du bâtiment reste largement en deçà du seuil de dangerosité pour la santé humaine, ne dépassant pas les -30 dBm.

Ainsi, il est possible de rester dans l'usine Meet-A sans avoir de soucis de santé et le dispositif garantit une connexion fiable pour les robots.

Des tests ont été réalisés avec un seul émetteur TX1 mais il restait toujours de petites zones où le débit était trop faible. Ci-dessous se trouvent des répartitions de débit avec un seul TX1.



(a) Répartition de débit internet en Mb/s avec un émetteur TX1



(b) Répartition de débit internet en Mb/s avec un émetteur TX1

4 Fonctionnalités du code et optimisations

Afin de simplifier le code et sa lecture, de nombreuses fonctions ont été créées. Chacune de ces fonctions sera décrite brièvement.

Commençons d'abord par les fonctions dessine_murs et dessine_rayons.
Pour pourvoir dessiner un mur, on n'a besoin que de deux points de ce mur : ses deux extrémités. Dans le code, un mur sera donc caractérisé par une matrice 2x3 : les coordonnées en x et y pour les 2 extrémités et 2 valeurs supplémentaires qui correspondent à leur permittivité relative et leur conductivité. Tous les murs sont stockés dans une seule liste. La fonction dessine_murs reprend simplement la liste de murs et dessine un segment (avec la fonction MATLAB line) partant d'une extrémité jusqu'à l'autre, et cela pour chaque mur de la liste.
La fonction dessine_rayons dessine aussi des segments mais ici c'est la trajectoire du rayon qui est dessinée. La fonction a besoin du point départ du rayon, du point d'arrivée et des points où le rayon se réfléchit à un mur pour pouvoir le dessiner.

Continuons avec la fonction nombre_rayons qui permet de calculer le nombre de rayons possibles en fonction du nombre de murs et du nombre de réflexions maximal choisi. Par exemple, si on a 3 murs et 3 réflexions, la fonction renvoie le vecteur : [1, 3, 6, 12]. On a 4 éléments car il y a 3 réflexions et il faut ajouter le cas direct. On a donc 1 rayon pour le cas direct, 3 rayons pour le cas à une réflexion car il y a 3 murs, 6 cas pour le cas à 2 réflexions car on à 3 possibilités pour la première réflexion et 2 pour la suivante (un mur ne se réfléchit pas 2 fois consécutives sur le même mur) : ça donne $3 * 2 = 6$. Le même raisonnement se poursuit pour 3 réflexions ($3 * 2 * 2 = 12$) et ainsi de suite.

Passons maintenant aux fonctions point_intersection et intersection. La première calcule simplement l'intersection de 2 droites dont l'une est définie par les 2 extrémités du mur et l'autre par 2 points dans le plan. Elle prend aussi en compte les cas où une des 2 droites est verticale (car sa pente serait infinie).
L'autre fonction, appelée intersection, vérifie si le point d'intersection trouvé avec la fonction précédente se trouve bien sur le mur et sur le segment qui relie les 2 points dans le plan. Elle revoie 1 si c'est le cas et 0 sinon.

Ensuite, il y a la fonction normale qui calcule simplement la normale à un mur à partir de ses deux extrémités en partant d'un produit scalaire nul (définition de la normale). La fonction prend en compte le cas particulier où le mur est horizontal (dans ce cas la normale est verticale). Elle s'assure aussi d'obtenir un normale de norme égale à un.

La fonction calcul_cosi calcule le sinus et le cosinus de l'angle entre la normale d'un mur et un segment défini par les 2 points de ses extrémités. Cela se fait facilement avec la définition du produit scalaire. Pour le sinus, il suffit d'utiliser $sin(\theta_i) = \sqrt{1 - cos(\theta_i)^2}$.

image_TX calcule l'image du point TX par rapport à un mur. Pour cela, on calcule la normale du mur et on regarde le point d'intersection de la normale du mur passant par TX, on sait donc que cette normale passe par TX et TX+normale. Ensuite on calcule le vecteur défini par TX et le point d'intersection que l'on vient de trouver. Il reste à soustraire 2 fois ce vecteur à TX

pour trouver son image par rapport au mur.

Les 2 fonctions combinaisons_n et combinaisons_rec sont essentielles pour trouver trouver toutes les combinaisons possibles de réflexions. La première sert uniquement à initialiser la matrice contenant les combinaisons pour un nombre de réflexions donné. Le nombre de colonnes correspond au nombre de réflexions et le nombre de lignes correspond aux nombre de rayons possibles pour ce nombre de réflexions (calculé avec nombre_rayons). L'autre fonction est récursive et remplit la matrice des combinaisons tout en s'assurant que 2 nombres consécutifs dans une soient différents (pas 2 réflexions successives sur le même mur). Chaque numéro correspond à l'indice du mur dans la liste.

Maintenant que l'on a les combinaisons de réflexions, il faut calculer les images de TX. C'est le but de la fonction images_nreflexions. A partir d'une ligne de la matrice des combinaisons, la fonction utilise image_TX pour trouver son image par rapport au mur décrit par son indice et le fait autant de fois qu'il y a de réflexions à partir de la dernière image trouvée. Toutes les images sont stockées dans une matrice avec en plus l'indice des murs à partir duquel on trouve l'image.

Une fois que les images sont calculées, il faut trouver les points où le rayon se reflète au mur grâce à la fonction trajectoire_rayons. Elle commence par prendre la dernière image de TX et calcule le point d'intersection entre le mur à partir duquel on a déterminé le dernier TX et le segment reliant le dernier TX et RX. On appellera ce point ptI (pour point d'intersection). Elle fait de même mais pour l'avant dernier mur à partir duquel on a déterminé l'avant dernier TX et pour le segment reliant l'avant dernier TX et le dernier ptI calculé. Ce processus se répète qu'à ce qu'on arrive à la première image de TX. On obtient une matrice contenant tous les points de réflexion du rayon et les indices des murs où ont lieu les réflexions. De plus la fonction vérifie si la trajectoire de rayon est possible en regardant si les points d'intersections calculés sont bien sur le mur et sur le segment défini par une image de TX et le point d'intersection précédent (ou RX si on n'a pas encore de point d'intersection).

Le prochain groupe de 3 fonctions : calcul_coefficient, réflexion et transmission sert à calculer les coefficients de réflexion et de transmission et les multiplie entre eux. Pour les réflexions, on calcule l'angle incident de chaque segment du rayon par rapport aux normales des murs où il y a réflexion à l'aide de calcul_cosi et puis on applique simplement la formule du coefficient de réflexion. Pour les transmissions c'est différent puisqu'on ne sait pas à l'avance combien de fois le rayon va traverser de murs. On prend donc chaque segment du rayon et on regarde pour chaque mur (sauf ceux qui contiennent l'extrémité du segment car là il y a réflexion, il peut y avoir transmission pour ces murs mais pour d'autres segments) si il y a intersection. Si c'est le cas, on calcule l'angle incident de chaque segment du rayon par rapport aux normales des murs où il y a transmission et on applique la formule du coefficient de transmission. Enfin, tous les coefficients sont multipliés pour avoir le coefficient global du rayon.

Finalement vient la fonction calcul_puissances qui va calculer les puissances reçues dans toute la pièce. La fonction boucle en x et en y pour déplacer le récepteur dans la pièce. Elle va aussi boucler sur le nombre de réflexions et le nombre de rayons par réflexion pour obtenir les coefficients qui permettront d'avoir les champs électriques et donc la puissance moyenne dans

une zone de 0.5x0.5m. Le cas direct est fait en dehors de la boucle puisque sans réflexion, il n'y a pas de points d'intersection. Elle prend en compte le gain de l'antenne TX3, exclut les puissances trop proches des sources et ne garde que la puissance maximale des émetteurs. Les puissances d'abord en watts sont finalement converties en dBm.

Il ne reste que le code principal où les constantes, les émetteurs et les murs sont définis. On y initialise aussi les matrices contentant les puissances, les débits binaires et on calcule le nombre de rayons. Ensuite, pour chaque émetteur, on utilise la fonction calcul_puissances pour remplir la matrice des puissances. Une fois cela fait on vérifie qu'on ne dépasse pas l'exposition maximales et puis on convertit ces puissances (en dBm) en débit binaire (Mb/s) (qu'on plafonne à 350Mb/s).

Dans le code, quelques optimisations peuvent être constatées.
Premièrement, la fonction combinaisons_rec est récursive, elle donne toutes les combinaisons possibles pour n'importe quel nombre de réflexions (sauf 0). Le code peut alors fonctionner pour autant de réflexions que l'on souhaite (même si ici on se limitera à 2 réflexions).

Ensuite, les coefficients pour le calcul de la puissance sont multipliés par les normes des champs au carré une fois que ceux-ci sont sommés et pas à chaque fois qu'on calcule le champ, les boucles for ont été parallélisées le mieux possible, le débit est calculé seulement une fois que la puissance est calculée.

Finalement, peu de fonctions trigonométriques ont été utilisées, en effet pour les angles d'incidence, la définition du produit scalaire a été utilisée à la place. La seule fois on une fonction trigonométrique à été utilisée est quand on a un émetteur TX3 et sert à calculer l'angle avec lequel partent les rayons. Ceci est inévitable car on a besoin de l'angle et non de son sinus ou cosinus.

5 Utilisation du GUI Matlab

L'interface graphique de Matlab offre l'avantage d'afficher facilement des graphiques en un seul clic. L'application que nous présenterons comprendra trois boutons, chacun permettant d'afficher différents types de graphiques. Le premier bouton affichera la répartition de la puissance en dBm, le deuxième bouton affichera la répartition du débit en Mb/s et le dernier bouton affichera les rayons émis par l'émetteur TX3. Il est également possible de placer le récepteur RX dont la position est choisie par l'utilisateur.

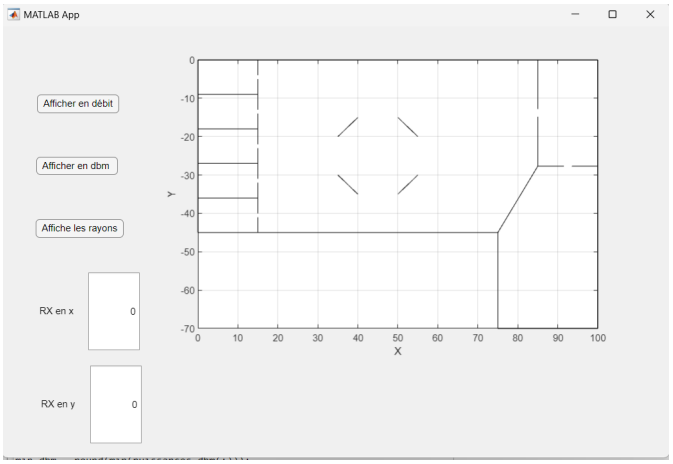


FIGURE 12 – Interface graphique Matlab

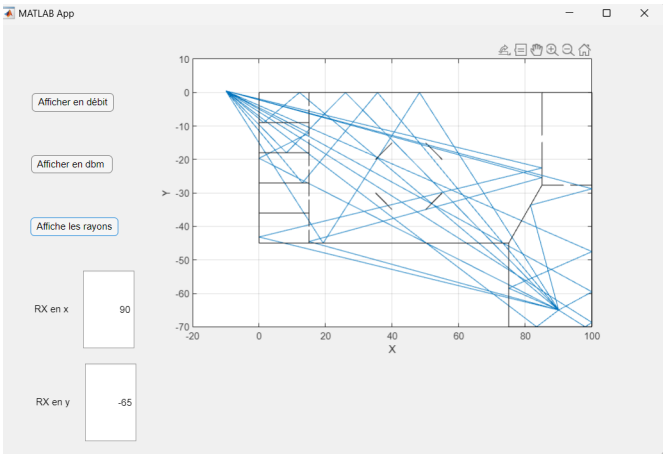


FIGURE 13 – Interface graphique Matlab affichant les rayons

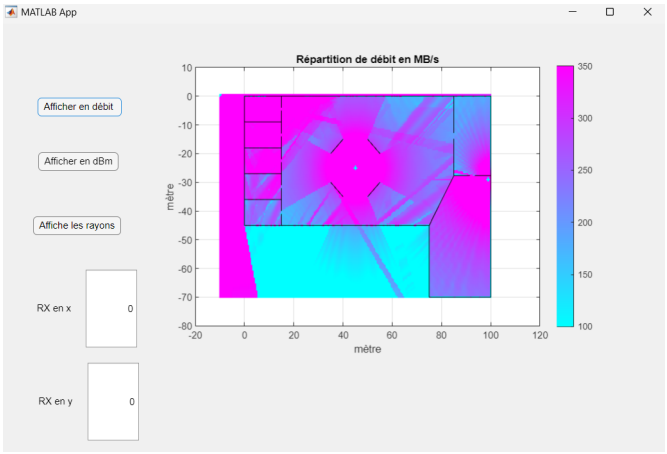


FIGURE 14 – Interface graphique Matlab affichant la répartition du débit en MB/s

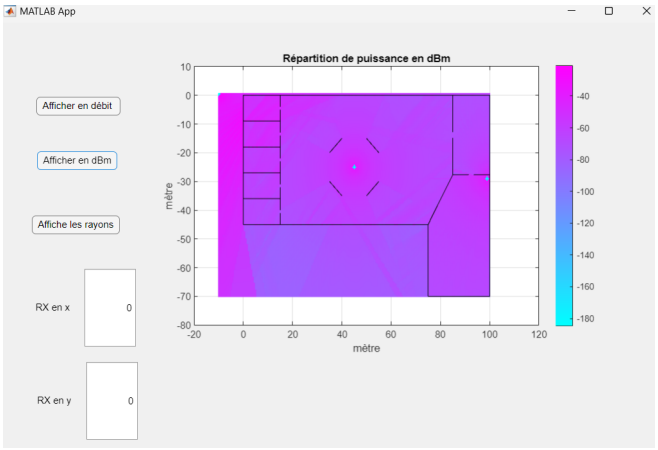


FIGURE 15 – Interface graphique Matlab affichant la répartition de puissance en dBm

A Annexe : Code

A.1 Code principal

```
clear; close; clc;

%% constantes for project

f = (26*10^9); % 5G 26GHz
omega = (2*pi*f);
epsilon0 = (8.85*10^-12);
mu0 = (4*pi*10^-7);
c = (3*10^8);
Z0 = sqrt(mu0/epsilon0);
beta = (omega/c);
lambda = (c/f);
Ra = 73;

%% constantes for tp8.1

% f = (868.3*10^6); % l'antenne du tp8
% omega = (2*pi*f);
% epsilon0 = (8.85*10^-12);
% mu0 = (4*pi*10^-7);
% c = (3*10^8);
% Z0 = sqrt(mu0/epsilon0);
% beta = (omega/c);
% lambda = (c/f);
% Ra = 73;


%% Emetteurs for projects
% TX1
puissanceTX1 = 20; %en dbm
gainTX1 = 1;
PtxGtxTX1 = gainTX1*10^((puissanceTX1-30)/10);
facteurTX1 = 60*lambda^2*PtxGtxTX1/(8*pi^2*Ra);

% TX2
puissanceTX2 = 35; %en dbm
gainTX2 = 1.7;
PtxGtxTX2 = gainTX2*10^((puissanceTX2-30)/10);
facteurTX2 = 60*lambda^2*PtxGtxTX2/(8*pi^2*Ra);

% TX3
puissanceTX3 = 35; %en dbm
gain_max = 21.5836; %en db
delta = -0.3; %en radians
phi_3db = (30*pi/180); %en radians
```

```
Ptx3 = 10^((puissanceTX3-30)/10);
facteurTX3 = 60*lambda^2*Ptx3/(8*pi^2*Ra);


%% Emetteurs for tp 8.1

%TX
 % PtxGtxTX = 60*1.64*10^-3;
 % facteurTX = lambda^2*PtxGtxTX/(8*pi^2*Ra);


%% Configuration de la piece for project

murs = {[[0;-45],[75;-45],[5;0.014]],[[75;-45],[75;-70],[5;0.014]],...
    [[75;-70],[100;-70],[5;0.014]],[[100;-70],[100;0],[4.6;0.02]],...
    [[100;0],[0;0],[4.6;0.02]],[[0;0],[0;-45],[4.6;0.02]],...
    [[15;0],[15;-4],[2.25;0.04]],[[15;-5],[15;-13],[2.25;0.04]],...
    [[15;-14],[15;-22],[2.25;0.04]],[[15;-23],[15;-31],[2.25;0.04]],...
    [[15;-32],[15;-40],[2.25;0.04]],[[15;-41],[15;-45],[2.25;0.04]],...
    [[0;-9],[15;-9],[2.25;0.04]],[[0;-18],[15;-18],[2.25;0.04]],...
    [[0;-27],[15;-27],[2.25;0.04]],[[0;-36],[15;-36],[2.25;0.04]],...
    [[35;-20],[40;-15],[2.25;0.04]],[[50;-15],[55;-20],[2.25;0.04]],...
    [[50;-35],[55;-30],[2.25;0.04]],[[35;-30],[40;-35],[2.25;0.04]],...
    [[85;0],[85;-12.8398],[4.6;0.02]],[[85;-14.8398],[85;-27.6796],[4.6;0.0
    [[75;-45],[85;-27.6796],[4.6;0.02]],[[85;-27.6796],[91.5;-27.6796],[4.6
    [[93.5;-27.6796],[100;-27.6796],[4.6;0.02]]};


nmurs = numel(murs); %nombre de murs
nrefl = 1; %nombre de reflexions
TX3 = [-10;0.5]; %fixé
n = 2; %nombre de pas par mètre
X = 100; %x max
Y = 0.5; %y max
Xm = -10; %x min
Ym = -70; %y min



%% configuration for tp8.1

% murs = {[[0;0], [0;80], [4.8;0.018]], [[0;20], [80;20], [4.8;0.018]], [[0
%
% nmurs = numel(murs); %nombre de murs
% nrefl = 2; %nombre de reflexions
% n = 2; %nombre de pas par mètre
% X = 80; %x max
% Y = 80; %y max
% Xm = 0; %x min
```

```
% Ym = 0; %y min
% TX = [32;10];

tic %mesure temps debut

nrayons = nombre_rayons(nmurs, nrefl); %calcul nombre rayons max
puissances_dbm = zeros(n*(Y-Ym)+1, n*(X-Xm)+1); %matrice des puissances
debit = zeros(n*(Y-Ym)+1, n*(X-Xm)+1); %matrice des débits
TXs = zeros(2,3);

%% TX for tp8.1

 % TXs(:,1) = TX;
 % k = 1;
 % RX = [47;65];
 % [Prx] = calcul_puissanceRX_TP8(murs, nmurs, nrayons, nrefl, TX, Z0, ...
 %    beta, omega, epsilon0, mu0,RX,facteurTX) ;
 % fprintf('%s Watt est la puissance en (47,65).\n',Prx)
 % [puissances_dbm] = calcul_puissances_TP8(murs, nmurs, nrayons, nrefl, TX
 %    beta, omega, epsilon0, mu0, puissances_dbm, facteurTX,TXs, k);



%% TX3
TXs(:,1) = TX3;
k = 1;
[puissances_dbm]  = calcul_puissances(murs, nmurs, nrayons, nrefl, TX3, n,
    beta, omega, epsilon0, mu0, puissances_dbm, 0, facteurTX2, gain_max, de

% TX1
TX1_1 = [45;-25];
TXs(:,2) = TX1_1;
k = 2;
[puissances_dbm]  = calcul_puissances(murs, nmurs, nrayons, nrefl, TX1_1, n
    beta, omega, epsilon0, mu0, puissances_dbm, 0, facteurTX1, gain_max, de

TX1_2 = [99;-29];
TXs(:,3) = TX1_2;
k = 3;
[puissances_dbm]  = calcul_puissances(murs, nmurs, nrayons, nrefl, TX1_2, n
    beta, omega, epsilon0, mu0, puissances_dbm, 0, facteurTX1, gain_max, de

%verification puissance maximale
for i=1:45*n
      for j=10*n:n*110 + 1
           if puissances_dbm(i,j) > -30
            disp('Erreur: exposition onde admise dépassée !')
          end
     end
```

```
end
for i=n*45+1:n*70+1
    for j=85*n:n*110 + 1
        if puissances_dbm(i,j) > -30
            disp('Erreur: exposition onde admise dépassée !')
        end
    end
end

 %calcul debit
  for ind_y=1:n*(Y-Ym)+1
      for ind_x=1:n*(X-Xm)+1
          debit(ind_y,ind_x) = 15*(puissances_dbm(ind_y,ind_x)+80)+50;
     end
  end

min_dbm = round(min(puissances_dbm(:)));
max_dbm = round(max(puissances_dbm(:)));
extremums = [100, 350]; %minimum = débit minimal que les robots puissent fo

hold on;
figure(1)
imagesc([Xm,X], [Ym,Y], debit, extremums); %dessine le graphe de la reparti
c = colorbar;
xlabel('mètre'); ylabel('mètre');c.Label.String = 'Débit [MB/s]';
set(gca,'YDir','normal') %sinon inversion axe Y
dessine_murs(murs); %dessine les murs
hold on;
figure(2)
imagesc([Xm,X], [Ym,Y], puissances_dbm, [-100, -30]); %dessine le graphe de
d = colorbar;
d.Label.String = 'Puissance[Dbm]';
xlabel('mètre'); ylabel('mètre');
set(gca,'YDir','normal') %sinon inversion axe Y
dessine_murs(murs); %dessine les murs


toc %mesure temps fin
```

A.2 Fonctions

```
function [Prx] = calcul_puissanceRX_TP8(murs, nmurs, nrayons, nrefl, TX, Z0
    beta, omega, epsilon0, mu0,RX,facteurTX)


      E = 0; %initialisation puissance

      %cas direct
      dist_tot = sqrt((TX(1)-RX(1))^2+(TX(2)-RX(2))^2); %distance parcourue
      coefficient = transmission(murs, nmurs, TX, RX, 0, 0, Z0, beta, omega
      E = E + coefficient*(exp(-1j*(beta)*dist_tot))/dist_tot;
      Prx = facteurTX*abs(E)^2;
      figure(3);
      hold on;
      dessine_murs(murs);
      hold on;
      dessine_rayons(RX, TX, TX, 0,Prx);
      xlabel('mètre'); ylabel('mètre');


            %cas avec reflexions

      for refl=1:nrefl %on boucle sur le nombre de reflexions
        comb_n = combinaisons_n(nmurs, nrayons(refl+1), refl); %combinaison
        for i=1:nrayons(refl+1) %trace tous les rayons et calcule les champ
          TXp = images_nreflexions(murs, comb_n(i,:), TX, refl); %calcul im

          [ptsI, rayon_poss] = trajectoire_rayons(murs, TXp, RX, refl); %do

          if rayon_poss == 1 %verifie si le rayon est possible
            TXn = TXp(1:2,refl); %derniere image de TX
            dist_tot = sqrt((TXn(1)-RX(1))^2+(TXn(2)-RX(2))^2); %distance p
            coefficient = calcul_coefficient(murs, nmurs, RX, ptsI, TX, ref
            display(dist_tot);
            if ~isnan(coefficient) %on enlève les cas ou le rayon se reflet
              E = E + coefficient*(exp(-1j*(beta)*dist_tot))/dist_tot;
              Prx_text = facteurTX*abs(E)^2;
              hold on;
              dessine_murs(murs);
              hold on;
              dessine_rayons(RX, ptsI, TX, refl,Prx_text);
              xlabel('mètre'); ylabel('mètre');
            end
          end
        end
      end

      Prx = facteurTX*abs(E)^2;
```

```
end


function [puissances_dbm] = calcul_puissances_TP8(murs, nmurs, nrayons, nre
    beta, omega, epsilon0, mu0, puissances_dbm, facteurTX,TXs, k)
  ind_y = 1;

  for y=n*Ym:n*Y
    ind_x = 1;
    for x=n*Xm:n*X
      RX = [(x/n);(y/n)];
      E = 0; %initialisation champ éléctrique

      %cas direct
      dist_tot = sqrt((TX(1)-RX(1))^2+(TX(2)-RX(2))^2); %distance parcourue
      coefficient = transmission(murs, nmurs, TX, RX, 0, 0, Z0, beta, omega
      E = E + coefficient*(exp(-1j*(beta)*dist_tot))/dist_tot;

      %cas avec reflexions

      for refl=1:nrefl %on boucle sur le nombre de reflexions
        comb_n = combinaisons_n(nmurs, nrayons(refl+1), refl); %combinaison
        for i=1:nrayons(refl+1) %trace tous les rayons et calcule les champ
          TXp = images_nreflexions(murs, comb_n(i,:), TX, refl); %calcul im
          [ptsI, rayon_poss] = trajectoire_rayons(murs, TXp, RX, refl); %do

          if rayon_poss == 1 %verifie si le rayon est possible
            TXn = TXp(1:2,refl); %derniere image de TX
            dist_tot = sqrt((TXn(1)-RX(1))^2+(TXn(2)-RX(2))^2); %distance p
            coefficient = calcul_coefficient(murs, nmurs, RX, ptsI, TX, ref
            if ~isnan(coefficient) %on enlève les cas ou le rayon se reflet
              E =  E +  coefficient*(exp(-1j*(beta)*dist_tot))/dist_tot;

            end
          end
        end
      end

      Prx = abs(E)^2;

      Prx = 30 + 10*log10(Prx*facteurTX); %dbm = 30+10*log(Prx);
      for u=1:k
        TXx = TXs(1,u);
        TXy = TXs(2,u);
        cercle_exclusion = ((x/n)-TXx)^2 + ((y/n)-TXy)^2;
        if cercle_exclusion <= (0.5)^2
          Prx = -inf;
        end
```

```
      end
      if puissances_dbm(ind_y,ind_x) == 0 || Prx == -inf %matrice vide
        puissances_dbm(ind_y,ind_x) = Prx;
      else
        puissances_dbm(ind_y,ind_x) = max(puissances_dbm(ind_y,ind_x), Prx)
      end
      ind_x = ind_x + 1;
    end
    ind_y = ind_y + 1;
  end
end



function coefficient = calcul_coefficient(murs, nmurs, ...
    RX, ptsI, TX, nrefl, Z0, beta, omega, epsilon0, mu0)
    coefficient = 1;

    %cas derniere reflexion
    mur = murs{ptsI(3,1)};
    gamma_mur = reflexion(mur, RX, ptsI(1:2,1), Z0, ...
        beta, omega, epsilon0, mu0);
    coefficient = coefficient*gamma_mur;

    %cas dernier segment et premier segment du rayon
    T0 = transmission(murs, nmurs, RX, ptsI(1:2,1), ptsI(3,1), ...
        ptsI(3,1), Z0, beta, omega, epsilon0, mu0);
    coefficient = coefficient*T0;
    T0 = transmission(murs, nmurs, ptsI(1:2,nrefl), TX, ...
        ptsI(3,nrefl), ptsI(3,nrefl), Z0, beta, omega, epsilon0, mu0);
    coefficient = coefficient*T0;

    for i=2:nrefl %cas segments intermediaires
        mur = murs{ptsI(3,i)};
        gamma_mur = reflexion(mur, ptsI(1:2,i-1), ptsI(1:2,i), Z0, ...
            beta, omega, epsilon0, mu0);
        coefficient = coefficient*gamma_mur;
        T0 = transmission(murs, nmurs, ptsI(1:2,i-1), ptsI(1:2,i), ...
            ptsI(3,i-1), ptsI(3,i), Z0, beta, omega, epsilon0, mu0);
        coefficient = coefficient*T0;
    end
end

function [cosi, sini] = calcul_cosi(mur, ptA, ptB)
%fonction calculant le cosinus du rayon incident
    normal = normale(mur);
    d = ptA-ptB;
    cosi = abs(d'*normal/sqrt(d(1)^2+d(2)^2));
    sini = sqrt(1-cosi^2);
```

```
end

function puissances_dbm = calcul_puissances(murs, nmurs, nrayons, ...
    nrefl, TX, n, Ym, Y, Xm, X, Z0, beta, omega, epsilon0, mu0, ...
    puissances_dbm, type, facteurTX, gain_max, delta, phi_3db, TXs, k)
  ind_y = 1;
  for y=n*Ym:n*Y
    ind_x = 1;
    for x=n*Xm:n*X
      RX = [(x/n);(y/n)];
      Prx = 0; %initialisation puissance

      %cas direct
      dist_tot = sqrt((TX(1)-RX(1))^2+(TX(2)-RX(2))^2);
      %distance parcourue par le rayon
      coefficient = transmission(murs, nmurs, TX, RX, 0, 0, Z0, ...
          beta, omega, epsilon0, mu0);
      E = coefficient*(exp(-1j*(beta)*dist_tot))/dist_tot;
      Prx = Prx + abs(E)^2;

      %cas avec reflexions
      for refl=1:nrefl %on boucle sur le nombre de reflexions
        comb_n = combinaisons_n(nmurs, nrayons(refl+1), refl);
        %combinaisons de reflexion
        for i=1:nrayons(refl+1)
          %trace tous les rayons et calcule les champs/coefficients
          TXp = images_nreflexions(murs, comb_n(i,:), TX, refl);
          %calcul images pour reflexions donnees
          [ptsI, rayon_poss] = trajectoire_rayons(murs, TXp, RX, refl);
          %donne la trajectoire d'un rayon

          if rayon_poss == 1 %verifie si le rayon est possible
            TXn = TXp(1:2,refl); %derniere image de TX
            dist_tot = sqrt((TXn(1)-RX(1))^2+(TXn(2)-RX(2))^2);
            %distance parcourue par le rayon
            coefficient = calcul_coefficient(murs, nmurs, RX, ptsI, TX, ...
                refl, Z0, beta, omega, epsilon0, mu0);
            if ~isnan(coefficient)
            %on enlève les cas ou le rayon se reflete 2 fois sur un coin
              E = coefficient*(exp(-1j*(beta)*dist_tot))/dist_tot;
              if type == 1 %antenne de type TX3
                vect = TX-ptsI(1:2,refl); %premier segment
                cos_phi = abs([1,0]*vect/sqrt(vect(1)^2+vect(2)^2));
                phi = acos(cos_phi);
                gain_theta = gain_max - 12*((phi-delta)/(phi_3db))^2;
                gainTX3 = 10^(gain_theta/10); %dB -> watts
                Prx = Prx + gainTX3*abs(E)^2;
              else %antennes type TX1 et TX2
                Prx = Prx + abs(E)^2;
```

```
            end
          end
        end
      end
    end

    Prx = 30 + 10*log10(Prx*facteurTX); %dbm = 30+10*log(Prx);
    for u=1:k
      TXx = TXs(1,u);
      TXy = TXs(2,u);
      cercle_exclusion = ((x/n)-TXx)^2 + ((y/n)-TXy)^2;
      if cercle_exclusion <= (0.5)^2
        Prx = -inf;
      end
    end
    if puissances_dbm(ind_y,ind_x) == 0 || Prx == -inf %matrice vide
      puissances_dbm(ind_y,ind_x) = Prx;
    else
      puissances_dbm(ind_y,ind_x) = max(puissances_dbm(ind_y, ...
          ind_x), Prx);
    end
    ind_x = ind_x + 1;
  end
  ind_y = ind_y + 1;
 end
end

function [comb, ind] = combinaisons_n(nmurs, nrayons, nrefl)
%fonction qui calcule les combinaisons de n réflexions
    ind = 1;
    i = 0;
    comb = zeros(nrayons, nrefl);
    vec = zeros(1,nrefl); %pour stocker les valeurs
    [comb, ind] = combinaisons_rec(comb, nmurs, nrefl, i, ind, vec);
end

function [comb, ind] = combinaisons_rec(comb, nmurs, nrefl, i, ind, vec)
%fonction qui calcule les combinaisons avec recursivite
    for j=1:nmurs
        if j ~= i
            if nrefl == 1
                vec(1,nrefl) = j;
                comb(ind,:) = vec;
                ind = ind+1;
            else
                vec(1,nrefl) = j;
                k = j;
                [comb, ind] = combinaisons_rec(comb, nmurs, ...
                    nrefl-1, k, ind, vec);
```

```
            end
        end
    end
end

function dessine_murs(murs)
%fonction qui dessine les murs à partir de leur liste
    nmurs = numel(murs);
    for i=1:nmurs
        line(murs{i}(1,1:2), murs{i}(2,1:2),'Color', 'black')
    end
end

function dessine_rayons(RX, ptI, TX, nrefl)
%fonction qui dessine les rayons
    for i=1:nrefl+1
        if i == 1
            line([RX(1),ptI(1,1)],[RX(2),ptI(2,1)])
        elseif i == nrefl+1
            line([ptI(1,i-1),TX(1)],[ptI(2,i-1),TX(2)])
        else
            line([ptI(1,i-1),ptI(1,i)],[ptI(2,i-1),ptI(2,i)])
        end
    end
end

function TXp = image_TX(TX, mur)
%calcule l'image d'un point TX par rapport à un mur
    normal = normale(mur);
    ptI = point_intersection(mur, TX, TX+normal);
    TXp = TX - 2*(TX-ptI);
end

function TXp = images_nreflexions(murs, comb_n, TX, nrefl)
%calcule les images de TX pour n reflexions
    TXp = zeros(3, nrefl);
    for i=1:nrefl
        ind_mur = comb_n(1,i);
        mur = murs{ind_mur};
        if i == 1
            TXp(:,i) = [image_TX(TX, mur); ind_mur];
        else
            TXp(:,i) = [image_TX(TXp(1:2,i-1), mur); ind_mur];
        end
    end
end

function inter = intersection(mur, ptI, ptA, ptB)
%détermine si le rayon et le mur s'intersectent
```

```
    xI = ptI(1);
    yI = ptI(2);
    xmmin = min(mur(1,1),mur(1,2)); %points extremaux du mur et du rayon
    xmmax = max(mur(1,1),mur(1,2));
    ymmin = min(mur(2,1),mur(2,2));
    ymmax = max(mur(2,1),mur(2,2));
    xpmin = min(ptA(1),ptB(1));
    xpmax = max(ptA(1),ptB(1));
    ypmin = min(ptA(2),ptB(2));
    ypmax = max(ptA(2),ptB(2));
    if (xmmin<=xI && xI<=xmmax && ymmin<=yI && yI<=ymmax)
        %on vérifie si le point d'intersection se trouve bien sur le mur
        %et sur le rayon
        if  (xpmin<=xI && xI<=xpmax && ypmin<=yI && yI<=ypmax)
            inter = 1;
        else
            inter = 0;
        end
    else
        inter = 0;
    end
end

function nrayons = nombre_rayons(nmurs, nrefl)
%calcule le nombre de rayons pour un nombre max de réflexions
    nrayons = zeros(1,nrefl);
    nrayons(1) = 1; %un rayon pour aucune réflexion
    for i=1:nrefl %i est le nombre de réflexions prises en compte
        nrayref = nmurs; %nombre de rayons pour un nombre de réflexions
        for j=1:i-1
            nrayref = nrayref*(nmurs-1);
        end
        nrayons(i+1) = nrayref;
    end
end

function normal = normale(mur) %fonction qui calcule la normale à un mur
   mx = mur(1,1)-mur(1,2);
   my = mur(2,1)-mur(2,2);
   if my == 0
       normal = [0;1];
   else
       ny = -mx/my;
       normal = [1;ny];
       normal = normal/sqrt(normal(1)^2+normal(2)^2);
    end
end

function ptI = point_intersection(mur, ptA, ptB)
```

```
%calcul de l'intersection de 2
%droites connaissant 2 points de chaque droite
    mx = mur(1,1)-mur(1,2);
    my = mur(2,1)-mur(2,2);
    px = ptA(1)-ptB(1);
    py = ptA(2)-ptB(2);
    if mx == 0 %mur vertical
        xI = mur(1,1);
        yI = py/px*(xI-ptA(1))+ptA(2); %calcul de y connaissant x
    elseif px == 0 %rayon vertical
        xI = ptA(1);
        yI = my/mx*(xI-mur(1,1))+mur(2,1);
    else %autres cas
        myx = my/mx; %pentes des droites
        pyx = py/px;
        xI = (ptA(2)-pyx*ptA(1)-mur(2,1)+myx*mur(1,1))/(myx-pyx);
        yI = my/mx*(xI-mur(1,1))+mur(2,1);
    end
    ptI = [xI;yI];
end

function gamma_mur = reflexion(mur, ptA, ptB, Z0, beta, omega, ...
    epsilon0, mu0)
%calcul coefficient de reflexion
    [cosi, sini] = calcul_cosi(mur, ptA, ptB);
    eps_r = mur(1,3);
    sint = (sqrt(1/eps_r)*sini);
    cost = sqrt(1-sint^2);
    conductivite = mur(2,3);
    k = (conductivite/omega);
    if eps_r == 4.6 %on regarde quelle est l'epaisseur du mur
        l = 0.3;
    elseif eps_r == 5
        l = 0.5;
    elseif eps_r == 2.25
        l = 0.1;
    end
    epsilon = (eps_r*epsilon0);
    eps_im = epsilon - 1i*(conductivite/omega);
    gamma = 1i*omega*sqrt(mu0*eps_im);
    Zm = sqrt(mu0/(epsilon-1j*k));
    s = (l/cost);
    gamma_per = (Zm*cosi-Z0*cost)/(Zm*cosi+Z0*cost); %perpendiculaire
    gamma_mur = gamma_per - (1-gamma_per^2)*(gamma_per*exp(-2*gamma*s)* ...
        exp(1j*beta*2*s*sint*sini))/(1-(gamma_per^2)*exp(-2*gamma*s)* ...
        exp(1j*beta*2*s*sint*sini));
end

function [ptsI, rayon_poss] = trajectoire_rayons(murs, TXp, RX, nrefl)
```

```
    ptsI = zeros(3, nrefl);
    for i=1:nrefl
        ind_mur = TXp(3,nrefl+1-i);
        mur = murs{ind_mur};
        if i == 1
            ptsI(:,i) = [point_intersection(mur, RX, ...
                TXp(1:2,nrefl)); ind_mur];
        else
            ptsI(:,i) = [point_intersection(mur, ptsI(1:2,i-1), ...
                TXp(1:2,nrefl+1-i)); ind_mur];
        end
    end
    rayon_poss = 1; %indique si la trajectoire du rayon est possible
    inter = intersection(murs{ptsI(3,1)}, ptsI(1:2,1), ...
        RX, TXp(1:2,nrefl));
    if inter == 0
        rayon_poss = 0;
    end
    for j=2:nrefl
        mur = murs{ptsI(3,j)};
        inter = intersection(mur, ptsI(1:2,j), ...
            ptsI(1:2,j-1), TXp(1:2,nrefl+1-j));
        if inter == 0
            rayon_poss = 0;
        end
    end
end

function T0 = transmission(murs, nmurs, ptA, ptB, ind1, ind2, Z0, ...
    beta, omega, epsilon0, mu0)
%calcul coefficient de transmission pour un segment entre 2 points
    T0 = 1;
    for i=1:nmurs
        if (i ~= ind1 && i ~= ind2) %on exclut les cas ou il y a reflexion
            mur = murs{i};
            eps_r = mur(1,3);
            conductivite = mur(2,3);
            k = (conductivite/omega);
            if eps_r == 4.6 %on regarde quelle est l'epaisseur du mur
                l = 0.3;
            elseif eps_r == 5
                l = 0.5;
            elseif eps_r == 2.25
                l = 0.1;
            end
            epsilon = (eps_r*epsilon0);
            eps_im = epsilon - 1i*(conductivite/omega);
            gamma = 1i*omega*sqrt(mu0*eps_im);
            Zm = sqrt(mu0/(epsilon-1j*k));
```

```
            ptI = point_intersection(mur, ptA, ptB);
            inter = intersection(mur, ptI, ptA, ptB);
            if inter == 1 %cas ou il y a transmission
                [cosi, sini] = calcul_cosi(mur, ptA, ptB);
                sint = sqrt(1/eps_r)*sini;
                cost = sqrt(1-sint^2);
                s = (l/cost);
                gamma_per = (Zm*cosi-Z0*cost)/(Zm*cosi+Z0*cost);
                T0p = ((1-gamma_per^2)*exp(-gamma*s))/(1-(gamma_per^2)* ...
                    exp(-2*gamma*s)*exp(1j*beta*2*s*sint*sini));
                T0 = T0*T0p;
            end
        end
    end
end
```

A.3 Code du GUI Matlab

```
classdef Interface < matlab.apps.AppBase

    % Properties that correspond to app components
    properties (Access = public)
        UIFigure                matlab.ui.Figure
        RXenyEditField          matlab.ui.control.NumericEditField
        RXenyEditFieldLabel     matlab.ui.control.Label
        RXenxEditField          matlab.ui.control.NumericEditField
        RXenxEditFieldLabel     matlab.ui.control.Label
        AffichelesrayonsButton  matlab.ui.control.Button
        AfficherendBmButton     matlab.ui.control.Button
        AfficherendbitButton_2  matlab.ui.control.Button
        UIAxes                  matlab.ui.control.UIAxes
    end


    properties (Access = public)

        RX1 = [45;-25];

        murs = {[[0;-45],[75;-45],[5;0.014]],[[75;-45],[75;-70],[5;0.014]],
    [[75;-70],[100;-70],[5;0.014]],[[100;-70],[100;0],[4.6;0.02]],...
    [[100;0],[0;0],[4.6;0.02]],[[0;0],[0;-45],[4.6;0.02]],...
    [[15;0],[15;-4],[2.25;0.04]],[[15;-5],[15;-13],[2.25;0.04]],...
    [[15;-14],[15;-22],[2.25;0.04]],[[15;-23],[15;-31],[2.25;0.04]],...
    [[15;-32],[15;-40],[2.25;0.04]],[[15;-41],[15;-45],[2.25;0.04]],...
    [[0;-9],[15;-9],[2.25;0.04]],[[0;-18],[15;-18],[2.25;0.04]],...
    [[0;-27],[15;-27],[2.25;0.04]],[[0;-36],[15;-36],[2.25;0.04]],...
    [[35;-20],[40;-15],[2.25;0.04]],[[50;-15],[55;-20],[2.25;0.04]],...
    [[50;-35],[55;-30],[2.25;0.04]],[[35;-30],[40;-35],[2.25;0.04]],...
```

```
    [[85;0],[85;-12.8398],[4.6;0.02]],[[85;-14.8398],[85;-27.6796],[4.6;0.0
    [[75;-45],[85;-27.6796],[4.6;0.02]],[[85;-27.6796],[91.5;-27.6796],[4.6
    [[93.5;-27.6796],[100;-27.6796],[4.6;0.02]]};

    end

   methods (Access = private)

       function [debit,puissances_dbm,X,Xm,Y,Ym]= main(app)


            %% constantes
f = (26*10^9); % 5G 26GHz
omega = (2*pi*f);
epsilon0 = (8.85*10^-12);
mu0 = (4*pi*10^-7);
c = (3*10^8);
Z0 = sqrt(mu0/epsilon0);
beta = (omega/c);
lambda = (c/f);
Ra = 73;

%% Emetteurs
% TX1
puissanceTX1 = 20; %en dbm
gainTX1 = 1;
PtxGtxTX1 = gainTX1*10^((puissanceTX1-30)/10);
facteurTX1 = 60*lambda^2*PtxGtxTX1/(8*pi^2*Ra);

% TX2
% puissanceTX2 = 35; %en dbm
% gainTX2 = 1;
% % PtxGtxTX2 = gainTX2*10^((puissanceTX2-30)/10);
% % facteurTX2 = 60*lambda^2*PtxGtxTX2/(8*pi^2*Ra);

% TX3
puissanceTX3 = 35; %en dbm
gain_max = 21.5836; %en db
delta = 0.75; %en radians
phi_3db = (30*pi/180); %en radians
Ptx3 = 10^((puissanceTX3-30)/10);
facteurTX3 = 60*lambda^2*Ptx3/(8*pi^2*Ra);

%% Configuration de la piece


nmurs = numel(app.murs); %nombre de murs
nrefl = 1; %nombre de reflexions
TX3 = [-10;0.5]; %fixé
```

```
n = 2; %nombre de pas par mètre
X = 100; %x max
Y = 0.5; %y max
Xm = -10; %x min
Ym = -70; %y min

tic %mesure temps debut

nrayons = nombre_rayons(app,nmurs, nrefl); %calcul nombre rayons max
puissances_dbm = zeros(n*(Y-Ym)+1, n*(X-Xm)+1); %matrice des puissances
debit = zeros(n*(Y-Ym)+1, n*(X-Xm)+1); %matrice des débits
TXs = zeros(2,3);


%% TX3
TXs(:,1) = TX3;
k = 1;
[puissances_dbm]  = calcul_puissances(app,app.murs, nmurs, nrayons, nrefl,
    beta, omega, epsilon0, mu0, puissances_dbm, 1, facteurTX3, gain_max, de

%% TX1
TX1_1 = [45;-25];
TXs(:,2) = TX1_1;
k = 2;
[puissances_dbm]  = calcul_puissances(app,app.murs, nmurs, nrayons, nrefl,
    beta, omega, epsilon0, mu0, puissances_dbm, 0, facteurTX1, gain_max, de

TX1_2 = [99;-29];
TXs(:,3) = TX1_2;
k = 3;
[puissances_dbm]  = calcul_puissances(app,app.murs, nmurs, nrayons, nrefl,
    beta, omega, epsilon0, mu0, puissances_dbm, 0, facteurTX1, gain_max, de

%verification puissance maximale
for i=1:45*n
      for j=10*n:n*110 + 1
           if puissances_dbm(i,j) > -30
            disp('Erreur: exposition onde admise dépassée !')
          end
     end
end
for i=n*45+1:n*70+1
    for j=85*n:n*110 + 1
        if puissances_dbm(i,j) > -30
            disp('Erreur: exposition onde admise dépassée !')
        end
    end
end
%calcul debit
```

```
for ind_y=1:n*(Y-Ym)+1
    for ind_x=1:n*(X-Xm)+1
        debit(ind_y,ind_x) = 15*(puissances_dbm(ind_y,ind_x)+80)+50;
    end
end


% hold on;
% imagesc(app.UIAxes,[Xm,X], [Ym,Y], puissances_dbm, [min_dbm, max_dbm]); %
% xlabel('mètre'); ylabel('mètre');
% set(gca,'YDir','normal') %sinon inversion axe Y
% dessine_murs(app,murs); %dessine les murs

toc %mesure temps fin
       end


    function affiche_rayons(app)



      nmurs = numel(app.murs); %nombre de murs

      nrefl = 0; %nombre de reflexions
      nrayons = nombre_rayons(app,nmurs, nrefl);
      f = (26*10^9); % 5G 26GHz
      omega = (2*pi*f);
      epsilon0 = (8.85*10^-12);
      mu0 = (4*pi*10^-7);
      c = (3*10^8);
      Z0 = sqrt(mu0/epsilon0);
      beta = (omega/c);
      TX = [-10;0.5]; %fixé



      dessine_murs(app,app.murs); %dessine les murs
      hold(app.UIAxes,'on')
      for refl=1:nrefl %on boucle sur le nombre de reflexions
        comb_n = combinaisons_n(app,nmurs, nrayons(refl+1), refl); %combina
        for i=1:nrayons(refl+1) %trace tous les rayons et calcule les champ
          TXp = images_nreflexions(app,app.murs, comb_n(i,:), TX, refl); %c
          [ptsI, rayon_poss] = trajectoire_rayons(app,app.murs, TXp, app.RX

          if rayon_poss == 1 %verifie si le rayon est possible

            coefficient = calcul_coefficient(app,app.murs, nmurs, app.RX1,
```

```
            if ~isnan(coefficient) %on enlève les cas ou le rayon se reflet

                dessine_rayons(app,app.RX1,ptsI,TX,refl)
                hold(app.UIAxes,'on')
            end
          end
        end
      end

      dessine_rayons(app,app.RX1,app.RX1,TX,1)
      hold(app.UIAxes,'on')




       end

       function T0 = transmission(app,murs, nmurs, ptA, ptB, ind1, ind2, Z0
%calcul coefficient de transmission pour un segment entre 2 points
    T0 = 1;
    for i=1:nmurs
        if (i ~= ind1 && i ~= ind2) %on exclut les cas ou il y a reflexion
            mur = murs{i};
            eps_r = mur(1,3);
            conductivite = mur(2,3);
            k = (conductivite/omega);
            if eps_r == 4.6 %on regarde quelle est l'epaisseur du mur
                l = 0.3;
            elseif eps_r == 5
                l = 0.5;
            elseif eps_r == 2.25
                l = 0.1;
            end
            epsilon = (eps_r*epsilon0);
            eps_im = epsilon - 1i*(conductivite/omega);
            gamma = 1i*omega*sqrt(mu0*eps_im);
            Zm = sqrt(mu0/(epsilon-1j*k));
            ptI = point_intersection(app,mur, ptA, ptB);
            inter = intersection(app,mur, ptI, ptA, ptB);
            if inter == 1 %cas ou il y a transmission
                [cosi, sini] = calcul_cosi(app,mur, ptA, ptB);
                sint = sqrt(1/eps_r)*sini;
                cost = sqrt(1-sint^2);
                s = (l/cost);
                gamma_per = (Zm*cosi-Z0*cost)/(Zm*cosi+Z0*cost);
                T0p = ((1-gamma_per^2)*exp(-gamma*s))/...
                    (1-(gamma_per^2)*exp(-2*gamma*s)*exp(1j*beta*2*s*sint*s
                T0 = T0*T0p;
```

```
            end
        end
    end
       end

       function [ptsI, rayon_poss] = trajectoire_rayons(app,murs, TXp, RX,
    ptsI = zeros(3, nrefl);
    for i=1:nrefl
        ind_mur = TXp(3,nrefl+1-i);
        mur = murs{ind_mur};
        if i == 1
            ptsI(:,i) = [point_intersection(app,mur, RX, TXp(1:2,nrefl)); i
        else
            ptsI(:,i) = [point_intersection(app,mur, ptsI(1:2,i-1), TXp(1:2
        end
    end
    rayon_poss = 1; %indique si la trajectoire du rayon est possible
    inter = intersection(app,murs{ptsI(3,1)}, ptsI(1:2,1), RX, TXp(1:2,nref
    if inter == 0
        rayon_poss = 0;
    end
    for j=2:nrefl
        mur = murs{ptsI(3,j)};
        inter = intersection(app,mur, ptsI(1:2,j), ptsI(1:2,j-1), TXp(1:2,n
        if inter == 0
            rayon_poss = 0;
        end
    end
       end

       function gamma_mur = reflexion(app,mur, ptA, ptB, Z0, beta, omega, e
%calcul coefficient de reflexion
    [cosi, sini] = calcul_cosi(app,mur, ptA, ptB);
    eps_r = mur(1,3);
    sint = (sqrt(1/eps_r)*sini);
    cost = sqrt(1-sint^2);
    conductivite = mur(2,3);
    k = (conductivite/omega);
    if eps_r == 4.6 %on regarde quelle est l'epaisseur du mur
        l = 0.3;
    elseif eps_r == 5
        l = 0.5;
    elseif eps_r == 2.25
        l = 0.1;
    end
    epsilon = (eps_r*epsilon0);
    eps_im = epsilon - 1i*(conductivite/omega);
    gamma = 1i*omega*sqrt(mu0*eps_im);
    Zm = sqrt(mu0/(epsilon-1j*k));
```

```
    s = (l/cost);
    gamma_per = (Zm*cosi-Z0*cost)/(Zm*cosi+Z0*cost); %perpendiculaire
    gamma_mur = gamma_per - (1-gamma_per^2)*(gamma_per*exp(-2*gamma*s)*exp(
        (1-(gamma_per^2)*exp(-2*gamma*s)*exp(1j*beta*2*s*sint*sini));
       end


       function ptI = point_intersection(~,mur, ptA, ptB)
%calcul de l'intersection de 2 droites connaissant 2 points de chaque droit
    mx = mur(1,1)-mur(1,2);
    my = mur(2,1)-mur(2,2);
    px = ptA(1)-ptB(1);
    py = ptA(2)-ptB(2);
    if mx == 0 %mur vertical
        xI = mur(1,1);
        yI = py/px*(xI-ptA(1))+ptA(2); %calcul de y connaissant x
    elseif px == 0 %rayon vertical
        xI = ptA(1);
        yI = my/mx*(xI-mur(1,1))+mur(2,1);
    else %autres cas
        myx = my/mx; %pentes des droites
        pyx = py/px;
        xI = (ptA(2)-pyx*ptA(1)-mur(2,1)+myx*mur(1,1))/(myx-pyx);
        yI = my/mx*(xI-mur(1,1))+mur(2,1);
    end
    ptI = [xI;yI];
       end

       function normal = normale(~,mur) %fonction qui calcule la normale no
   mx = mur(1,1)-mur(1,2);
   my = mur(2,1)-mur(2,2);
   if my == 0
       normal = [0;1];
   else
       ny = -mx/my;
       normal = [1;ny];
       normal = normal/sqrt(normal(1)^2+normal(2)^2);
    end
       end

       function nrayons = nombre_rayons(~,nmurs, nrefl)
%calcule le nombre de rayons pour un nombre max de réflexions
    nrayons = zeros(1,nrefl);
    nrayons(1) = 1; %un rayon pour aucune réflexion
    for i=1:nrefl %i est le nombre de réflexions prises en compte
        nrayref = nmurs; %nombre de rayons pour un nombre donné de réflexio
        for j=1:i-1
            nrayref = nrayref*(nmurs-1);
        end
```

```
        nrayons(i+1) = nrayref;
    end
       end

       function inter = intersection(~,mur, ptI, ptA, ptB) %détermine si le
    xI = ptI(1);
    yI = ptI(2);
    xmmin = min(mur(1,1),mur(1,2)); %points extremaux du mur et du rayon
    xmmax = max(mur(1,1),mur(1,2));
    ymmin = min(mur(2,1),mur(2,2));
    ymmax = max(mur(2,1),mur(2,2));
    xpmin = min(ptA(1),ptB(1));
    xpmax = max(ptA(1),ptB(1));
    ypmin = min(ptA(2),ptB(2));
    ypmax = max(ptA(2),ptB(2));
    if (xmmin<=xI && xI<=xmmax && ymmin<=yI && yI<=ymmax)
        %on vérifie si le point d'intersection se trouve bien sur le mur
        if  (xpmin<=xI && xI<=xpmax && ypmin<=yI && yI<=ypmax) %et sur le r
            inter = 1;
        else
            inter = 0;
        end
    else
        inter = 0;
    end
       end

       function TXp = images_nreflexions(app,murs, comb_n, TX, nrefl)
%calcule les images de TX pour n reflexions
    TXp = zeros(3, nrefl);
    for i=1:nrefl
        ind_mur = comb_n(1,i);
        mur = murs{ind_mur};
        if i == 1
            TXp(:,i) = [image_TX(app,TX, mur); ind_mur];
        else
            TXp(:,i) = [image_TX(app,TXp(1:2,i-1), mur); ind_mur];
        end
    end
       end


       function TXp = image_TX(app,TX, mur) %calcule l'image d'un point TX
    normal = normale(app,mur);
    ptI = point_intersection(app,mur, TX, TX+normal);
    TXp = TX - 2*(TX-ptI);
       end

       function dessine_rayons(app,RX, ptI, TX, nrefl) %fonction qui dessin
```

```
    for i=1:nrefl+1
        if i == 1
            line(app.UIAxes,[RX(1),ptI(1,1)],[RX(2),ptI(2,1)])
        elseif i == nrefl+1
            line(app.UIAxes,[ptI(1,i-1),TX(1)],[ptI(2,i-1),TX(2)])
        else
            line(app.UIAxes,[ptI(1,i-1),ptI(1,i)],[ptI(2,i-1),ptI(2,i)])
        end
    end
       end

       function dessine_murs(app,murs) %fonction qui dessine les murs à par
    nmurs = numel(murs);
    for i=1:nmurs
        line(app.UIAxes,murs{i}(1,1:2), murs{i}(2,1:2),'Color', 'black')
    end
       end

       function [comb, ind] = combinaisons_rec(app,comb, nmurs, nrefl, i, i
%fonction qui calcule les combinaisons avec recursivite
    for j=1:nmurs
        if j ~= i
            if nrefl == 1
                vec(1,nrefl) = j;
                comb(ind,:) = vec;
                ind = ind+1;
            else
                vec(1,nrefl) = j;
                k = j;
                [comb, ind] = combinaisons_rec(app,comb, nmurs, nrefl-1, k,
            end
        end
    end
       end

       function [comb, ind] = combinaisons_n(app,nmurs, nrayons, nrefl)
%fonction qui calcule les combinaisons de n réflexions
    ind = 1;
    i = 0;
    comb = zeros(nrayons, nrefl);
    vec = zeros(1,nrefl); %pour stocker les valeurs
    [comb, ind] = combinaisons_rec(app,comb, nmurs, nrefl, i, ind, vec);
       end

       function [puissances_dbm] = calcul_puissances(app,murs, nmurs, nrayo
    beta, omega, epsilon0, mu0, puissances_dbm, type, facteurTX, gain_max,
  ind_y = 1;
  for y=n*Ym:n*Y
    ind_x = 1;
```

```
for x=n*Xm:n*X
  RX = [(x/n);(y/n)];
  Prx = 0; %initialisation puissance

  %cas direct
  dist_tot = sqrt((TX(1)-RX(1))^2+(TX(2)-RX(2))^2); %distance parcourue
  coefficient = transmission(app,murs, nmurs, TX, RX, 0, 0, Z0, beta, o
  E = coefficient*(exp(-1j*(beta)*dist_tot))/dist_tot;
  Prx = Prx + abs(E)^2;
  %cas avec reflexions

  for refl=1:nrefl %on boucle sur le nombre de reflexions
    comb_n = combinaisons_n(app,nmurs, nrayons(refl+1), refl); %combina
    for i=1:nrayons(refl+1) %trace tous les rayons et calcule les champ
      TXp = images_nreflexions(app,murs, comb_n(i,:), TX, refl); %calcu
      [ptsI, rayon_poss] = trajectoire_rayons(app,murs, TXp, RX, refl);

      if rayon_poss == 1 %verifie si le rayon est possible
        TXn = TXp(1:2,refl); %derniere image de TX
        dist_tot = sqrt((TXn(1)-RX(1))^2+(TXn(2)-RX(2))^2); %distance p
        coefficient = calcul_coefficient(app,murs, nmurs, RX, ptsI, TX,
        if ~isnan(coefficient) %on enlève les cas ou le rayon se reflet
          E = coefficient*(exp(-1j*(beta)*dist_tot))/dist_tot;
          if type == 1 %antenne de type TX3
            vect = TX-ptsI(1:2,refl); %premier segment
            cos_phi = abs([1,0]*vect/sqrt(vect(1)^2+vect(2)^2));
            phi = acos(cos_phi);
            gain_theta = gain_max - 12*((phi-delta)/(phi_3db))^2; %en d
            gainTX3 = 10^(gain_theta/10);
            Prx = Prx + gainTX3*abs(E)^2;
          else %antennes type TX1 et TX2
            Prx = Prx + abs(E)^2;
          end
        end
      end
    end
  end

  Prx = 30 + 10*log10(Prx*facteurTX); %dbm = 30+10*log(Prx);
  for u=1:k
    TXx = TXs(1,u);
    TXy = TXs(2,u);
    cercle_exclusion = ((x/n)-TXx)^2 + ((y/n)-TXy)^2;
    if cercle_exclusion <= (0.5)^2
      Prx = -inf;
    end
  end
  if puissances_dbm(ind_y,ind_x) == 0 || Prx == -inf %matrice vide
    puissances_dbm(ind_y,ind_x) = Prx;
```

```
      else
        puissances_dbm(ind_y,ind_x) = max(puissances_dbm(ind_y,ind_x), Prx)
      end
      ind_x = ind_x + 1;
    end
    ind_y = ind_y + 1;
  end
       end

       function [cosi, sini] = calcul_cosi(app,mur, ptA, ptB)
%fonction calculant le cosinus du rayon incident
    normal = normale(app,mur);
    d = ptA-ptB;
    cosi = abs(d'*normal/sqrt(d(1)^2+d(2)^2));
    sini = sqrt(1-cosi^2);
       end

       function coefficient = calcul_coefficient(app,murs, nmurs, ...
    RX, ptsI, TX, nrefl, Z0, beta, omega, epsilon0, mu0)
    coefficient = 1;

    %cas derniere reflexion
    mur = murs{ptsI(3,1)};
    gamma_mur = reflexion(app,mur, RX, ptsI(1:2,1), Z0, beta, omega, epsilo
    coefficient = coefficient*gamma_mur;

    %cas dernier segment et premier segment du rayon
    T0 = transmission(app,murs, nmurs, RX, ptsI(1:2,1), ptsI(3,1), ...
        ptsI(3,1), Z0, beta, omega, epsilon0, mu0);
    coefficient = coefficient*T0;
    T0 = transmission(app,murs, nmurs, ptsI(1:2,nrefl), TX, ...
        ptsI(3,nrefl), ptsI(3,nrefl), Z0, beta, omega, epsilon0, mu0);
    coefficient = coefficient*T0;

    for i=2:nrefl %cas segments intermediaires
        mur = murs{ptsI(3,i)};
        gamma_mur = reflexion(app,mur, ptsI(1:2,i-1), ptsI(1:2,i), Z0, beta
        coefficient = coefficient*gamma_mur;
        T0 = transmission(app,murs, nmurs, ptsI(1:2,i-1), ptsI(1:2,i), ...
            ptsI(3,i-1), ptsI(3,i), Z0, beta, omega, epsilon0, mu0);
        coefficient = coefficient*T0;
    end
       end

    end


    % Callbacks that handle component events
    methods (Access = private)
```

```
        % Button pushed function: AfficherendbitButton_2
        function AfficherendbitButton_2Pushed(app, event)

          [debit,~,X,Xm,Y,Ym]= main(app);

extremums = [100, 350]; %minimum = débit minimal que les robots puissent fo
hold(app.UIAxes,'on');
imagesc(app.UIAxes,[Xm,X], [Ym,Y], debit, extremums); %dessine le graphe de
xlabel(app.UIAxes,'mètre'); ylabel(app.UIAxes,'mètre');title(app.UIAxes,'Ré
dessine_murs(app,app.murs); %dessine les murs
hold(app.UIAxes,'on');
        end

        % Button pushed function: AfficherendBmButton
        function AfficherendBmButtonPushed(app, event)

            [~,puissances_dbm,X,Xm,Y,Ym]= main(app);

min_dbm = round(min(puissances_dbm(:)));
max_dbm = round(max(puissances_dbm(:)));
hold(app.UIAxes,'on');
imagesc(app.UIAxes,[Xm,X], [Ym,Y], puissances_dbm, [min_dbm, max_dbm]); %de
xlabel(app.UIAxes,'mètre'); ylabel(app.UIAxes,'mètre');title(app.UIAxes,'Ré
dessine_murs(app,app.murs); %dessine les murs
hold(app.UIAxes,'on');
        end

        % Button pushed function: AffichelesrayonsButton
        function AffichelesrayonsButtonPushed(app, event)

            affiche_rayons(app);

        end

        % Value changed function: RXenxEditField
        function RXenxEditFieldValueChanged(app, event)
            app.RX1(1) = event.Value;

        end

        % Value changed function: RXenyEditField
        function RXenyEditFieldValueChanged(app, event)
            app.RX1(2) = event.Value;

        end

        % Button down function: UIAxes
        function UIAxesButtonDown(app, event)
```

```
            dessine_murs(app,app.murs);
        end

        % Callback function
        function CleanButtonPushed(app, event)
            delete(affiche_rayons(app));
        end
    end

    % Component initialization
    methods (Access = private)

        % Create UIFigure and components
        function createComponents(app)

            % Create UIFigure and hide until all components are created
            app.UIFigure = uifigure('Visible', 'off');
            colormap(app.UIFigure, 'winter');
            app.UIFigure.Position = [100 100 833 537];
            app.UIFigure.Name = 'MATLAB App';

            % Create UIAxes
            app.UIAxes = uiaxes(app.UIFigure);
            xlabel(app.UIAxes, 'X')
            ylabel(app.UIAxes, 'Y')
            zlabel(app.UIAxes, 'Z')
            app.UIAxes.Box = 'on';
            app.UIAxes.XGrid = 'on';
            app.UIAxes.YGrid = 'on';
            app.UIAxes.ColorOrder = [0 0.450980392156863 0.741176470588235;
            app.UIAxes.NextPlot = 'add';
            app.UIAxes.ButtonDownFcn = createCallbackFcn(app, @UIAxesButton
            colormap(app.UIAxes, 'winter')
            app.UIAxes.Position = [205 125 551 379];

            % Create AfficherendbitButton_2
            app.AfficherendbitButton_2 = uibutton(app.UIFigure, 'push');
            app.AfficherendbitButton_2.ButtonPushedFcn = createCallbackFcn(
            app.AfficherendbitButton_2.Position = [49 422 105 23];
            app.AfficherendbitButton_2.Text = 'Afficher en débit ';

            % Create AfficherendBmButton
            app.AfficherendBmButton = uibutton(app.UIFigure, 'push');
            app.AfficherendBmButton.ButtonPushedFcn = createCallbackFcn(app
            app.AfficherendBmButton.Position = [50 354 100 23];
            app.AfficherendBmButton.Text = 'Afficher en dBm';

            % Create AffichelesrayonsButton
            app.AffichelesrayonsButton = uibutton(app.UIFigure, 'push');
```

```
            app.AffichelesrayonsButton.ButtonPushedFcn = createCallbackFcn(
            app.AffichelesrayonsButton.Position = [43 274 110 23];
            app.AffichelesrayonsButton.Text = 'Affiche les rayons';

            % Create RXenxEditFieldLabel
            app.RXenxEditFieldLabel = uilabel(app.UIFigure);
            app.RXenxEditFieldLabel.HorizontalAlignment = 'right';
            app.RXenxEditFieldLabel.Position = [43 172 51 22];
            app.RXenxEditFieldLabel.Text = 'RX en x ';

            % Create RXenxEditField
            app.RXenxEditField = uieditfield(app.UIFigure, 'numeric');
            app.RXenxEditField.ValueChangedFcn = createCallbackFcn(app, @RX
            app.RXenxEditField.Position = [109 134 64 97];

            % Create RXenyEditFieldLabel
            app.RXenyEditFieldLabel = uilabel(app.UIFigure);
            app.RXenyEditFieldLabel.HorizontalAlignment = 'right';
            app.RXenyEditFieldLabel.Position = [45 56 51 22];
            app.RXenyEditFieldLabel.Text = 'RX en y ';

            % Create RXenyEditField
            app.RXenyEditField = uieditfield(app.UIFigure, 'numeric');
            app.RXenyEditField.ValueChangedFcn = createCallbackFcn(app, @RX
            app.RXenyEditField.Position = [111 18 64 97];

            % Show the figure after all components are created
            app.UIFigure.Visible = 'on';
        end
    end

    % App creation and deletion
    methods (Access = public)

        % Construct app
        function app = Interface

            % Create UIFigure and components
            createComponents(app)

            % Register the app with App Designer
            registerApp(app, app.UIFigure)

            if nargout == 0
                clear app
            end
        end

        % Code that executes before app deletion
```

```
        function delete(app)

            % Delete UIFigure when app is deleted
            delete(app.UIFigure)
        end
    end
end
```